SUP'COM

المدرسة العليا للمواصلات بتونس

ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Telecommunications Engineers Training Cycle

Option

Cybersecurity and Defense

End of Study Project Report

Topic: Design and implementation of a Sybil attack detection algorithm in ITS

AUTHOR:

Dhia Eddine Laouiti

LABORATORY SUPERVISORS:

Mr. Marwane Ayaida & Mr. Messai Nadhir

SCHOOL SUPERVISORS:

Mme Najeh Sameh & Mme Najjar Leila

ACADEMIC YEAR:

2020-2021

Ecole Supérieure des Communications de Tunis

SUP'COM

2083 Cité Technologique des Communications - Elghazala - Ariana - Tunisie

Tél. +216 70 240 900 – site web : www.supcom.mincom.tn

Acknowledgements

This report is the result of a pleasant graduation internship in research and development. That pleasure I owe to the interaction I have had with my supervisors, colleagues, and co-workers as I am delighted to express my gratitude to all those who have supported, guided, and helped me in some way during this project.

Special appreciation to my supervisors in the CReSTIC lab Mr. Marwane Ayaida and Mr. Messai Nadhir, and in SUP'COM Mme. Najeh Sameh & Mme. Najjar Leila for their enthusiastic encouragement and helpful advice, they have provided me during my internship. They are always carefully listening to my question, and guiding me to resolve the encountered problems.

I would also like to express my deepest gratitude to all my teachers for their training during my study years. I direct my very profound recognition to my friends and family for their support and continuous encouragement throughout my years of study and during this internship. I am glad to announce that this accomplishment would not have been possible without them.

Finally, I would like to thank the board members of the jury for their attention, and I hope they will find in this report the expected clarity and motivation.

Thank you all.

Abstract

Smart city is a wide range of projects to facilitate the problems of everyday life and ensure security, thus ITS takes care of the transportation issues using the VANET project which is a promising technology for autonomous driving providing many benefits and user conveniences to improve road safety and driving comfort.

The problem is with such rapid improvement and development continuously increasing the digital threats and mainly the Sybil attack is one of the most serious threats in vehicular communications because attackers can generate multiple forged identities to disseminate false messages to disrupt safety-related services or misuse the systems.

To address such an issue, we propose a behavior-based technique that is tested on the provided VeReMi dataset coupled with various machine learning techniques for comparison. The results prove the ability of the proposed mechanism to identify the attacks in VANET.

Key words: Intelligent Transport System, Vehicular Ad-Hoc Network Security, Sybil attack, Machine learning, Misbehavior detection, Vehicle Driving Pattern, VeReMi dataset.

Table of contents

Acknowledgements ***1***

Abstract ***2***

Table of contents ***3***

List of Figures ***6***

Glossary ***9***

General introduction ***10***

1. Chapter 1: Project Context and Specifications ***11***

1.1. Introduction **11**

1.2. Project Host **11**

1.3. Project Presentation **12**

1.3.1. Scope 12

1.3.2. General Concepts 12

1.3.2.1. Intelligent Transport System (ITS) 12

1.3.2.2. Vehicular Ad-Hoc Network(VANET) 13

1.3.2.2.1. Presentation of the VANET 13

1.3.2.2.2. VANET architecture 14

1.3.2.2.3. VANET characteristics 15

1.3.2.2.4. VANET challenges: 16

1.3.2.3. Attacker models 16

1.3.2.4. Attack examples 17

1.4. Conclusion **21**

2. Chapter 2: State of the art ***23***

2.1. Introduction **23**

2.2. Sybil attack presentation **23**

2.2.1. Sybil attack properties 23

2.2.2. Sybil attack effects: 25

2.2.3. Detection techniques 26

2.3. Related researches **27**

2.3.1. Proposed researches 27

2.3.2. Comparison: proposed techniques of previous researches 32

2.4. Conclusion **33**

3. Chapter 3: Proposed methodology and ML methods ***34***

3.1. Introduction: **34**

3.2. Proposed preprocessing techniques: **34**

3.2.1. The Matrix of movement 34

3.2.2. Modeling techniques 37

3.2.2.1. Under sampling ... 37
3.2.2.1.1. Random Under-Sampling ... 37
3.2.2.1.2. Tomek Links for Under sampling ... 37
3.2.2.1.3. Edited Nearest Neighbors Rule for Under sampling ... 38
3.2.2.2. Combined sampling ... 38
3.3. Machine learning methods ... 38
3.3.1. XGBoost ... 38
3.3.2. ELM ... 39
3.3.3. AdaBoost ... 39
3.3.4. KNN ... 40
3.3.5. Decision Tree ... 40
3.3.6. Random forest ... 40
3.4. Scoring methods: ... 40
3.4.1. Confusion Matrix ... 40
3.4.2. Accuracy ... 41
3.4.3. Precision ... 41
3.4.4. Recall ... 42
3.4.5. F1-score ... 42
3.4.6. ROC curve ... 42
3.5. Conclusion ... 43
4. Chapter 4: Offline results ... 44
4.1. Introduction ... 44
4.2. VeReMi dataset presentation: ... 44
4.2.1. Simulation Platform ... 44
4.2.2. VeReMi dataset: Feature study ... 45
4.2.2.1. Existent features ... 45
4.2.2.2. Added features ... 47
4.2.2.2.1. Prediction of the position ... 48
4.2.2.2.2. Acceleration variation ... 48
4.2.2.2.3. Predicted position plausibility: ... 49
4.3. ML offline results ... 50
4.3.1. Implementation of the Machine learning method ... 51
4.3.1.1. First Step: Choosing ML method ... 51
4.3.1.2. Second Step: Comparing in between lists ... 52
4.3.1.3. Third step: Improving the results ... 53
4.3.2. Comparison: Brut dataset & dataset with preprocessing ... 55
4.3.2.1. Results comparison: F1-score and confusion matrix ... 55
4.3.2.2. Advantages of proposed method ... 57
4.3.2.3. Disadvantages of proposed method ... 57
4.4. Conclusion ... 57
5. Chapter 5: Online Results ... 59
5.1. Introduction ... 59
5.2. Setting up the environment ... 59
5.2.1. Application structure ... 59
5.2.2. Preparing the environment ... 61
5.2.3. Sample presentation ... 64

5.2.3.1. Chosen scenario 64
5.2.3.2. Detection results 67
5.2.4. Proposed work 69
5.2.4.1. Misbehavior Authority server presentation 69
5.2.4.2. Work Implementation 69
5.3. Conclusion 72
General Conclusion 73
References 75

List of Figures

Figure 1: VANET connection possibilities 13
Figure 2:Vehicle node structure 14
Figure 3: Example of denial of service attack 18
Figure 4: Example of black hole attack 18
Figure 5: Example of Malware attack 19
Figure 6: Example of information modification 19
Figure 7: Example of message interruption and deletion 20
Figure 8: Example of eavesdropping 20
Figure 9: Example of impersonation 21
Figure 10: Example of a Sybil attack 21
Figure 11: Sybil attack 24
Figure 12: Categorization of Sybil defense techniques 26
Figure 13: Proposed model construction phases (ANN in VANET) 27
Figure 14: Proposed architecture of the SDVC 28
Figure 15: Example of a situation using this approach 31
Figure 16: Behavior matrix presentation 35
Figure 17: Form of a diagonal matrix 3x3 35
Figure 18: Proposed algorithm to calculate eigenvalues 36
Figure 19: ELM hidden network (scenario of 2 outputs) 39
Figure 20: Confusion matrix (In case of only two classes) 41
Figure 21: Example of ROC curve 42
Figure 22: Simulation Scenario 44
Figure 23: The features of VeReMi's Sybil attack dataset 47
Figure 24: Algorithm for position prediction 48
Figure 25: Algorithm for plausibility factor of the original dataset 49
Figure 26: Algorithm for average plausibility calculation 50
Figure 27: Histogram of F1-score & Accuracy using different ML algorithm 51
Figure 28: The confusion matrices of each ML algorithm in percent 51
Figure 29: Accuracy and F1-score comparison between both lists 52
Figure 30: First list confusion matrices 53
Figure 31: Second List confusion matrices 53
Figure 32: comparison between modeling techniques 53
Figure 33: Confusion matrix of most modeling techniques 54
Figure 34:results after adding the plausibility feature 54
Figure 35: Confusion matrices of modeling techniques 55
Figure 36: Evolution of ROC curve 55
Figure 37: Comparison between both datasets 56
Figure 38: Comparison between confusion matrices 56
Figure 39: ROC curves of both datasets 57
Figure 40: Relation between SUMO, OMNeT++ and VEINS 59
Figure 41: Example of OMNeT++ scenario 60
Figure 42: SUMO needed dependencies 62
Figure 43: SUMO package extraction 62

Figure 44: The used commands for SUMO installation 62
Figure 45: Environment variable of sumo in bash file 62
Figure 46: SUMO files name change 63
Figure 47: Installing dependencies for OMNeT++ 63
Figure 48: OMNeT files after extraction 63
Figure 49: Setting the environment then installing OMNeT++ 63
Figure 50: Github cloning of F²MD repository 64
Figure 51: F²MD installation command 64
Figure 52: Scenario of Paris-Saclay files 64
Figure 53: Editing the omentp.ini file 64
Figure 54: Reporting parameters of omnetp.ini file 65
Figure 55: Attacking parameters of omnetp.ini file 65
Figure 56: Launching the Traci Daemon bash program 66
Figure 57: Launching the misbehavior detection server bash program 66
Figure 58: Output of misbehavior detection on local server port 9980 66
Figure 59: Launching the Paris-Saclay scenario using IEEE protocol and CMdenv 66
Figure 60: SUMO Simulation graphic output 67
Figure 61: Misbehavior detection server output 68
Figure 62: Reaction of ongoing attack reports 68
Figure 63: The VeReMi dataset 68
Figure 64: Pseudo algorithm of the prediction 70
Figure 65: Misbehavior authority serve performance (first case) 71
Figure 66: Confusion Matrix and performance metrics(first case) 71
Figure 67: Misbehavior Authority performance (second case) 71
Figure 68: Confusion Matrix and performance metrics(second case) 72

List of tables

Tableau 1: RSU messages of VANET nodes 30
Tableau 2:Comparison between used time for model construction, training and prediction .. 57

Glossary

Abbreviation	Complete Term
ITS	Intelligent Transport System
VANET	Vehicular Ad-hoc Network
MANET	Mobile Ad-hoc Network
V2V	Vehicle to Vehicle
V2I	Vehicle to Infrastructure
RREQ	Route Request
RREP	Route Reply
RERR	Route Error
AU	Application Unit
OBU	On Board Unit
DSRC	Dedicated Short Range Communication
RSU	Road Side Unit
SDVC	Software-Defined Vehicular Cloud
ML	Machine Learning
KNN	K Nearest Neighbor
NNR	Nearest Neighbor rule
ANN	Artificial Neural Network
ELM	Extreme Learning Machine
VeReMi	Vehicular Reference Misbehavior

General introduction

The use of cars is in an exponential increasing rate to the point where it is obvious to say that the number of cars is dominating the world, as a result, we notice the increasing number of road accidents and their causes varies from the driver's fault (distraction, poor driving skills...) to the environment problem (weather, bad road condition...), thus to regulate this complication the researchers sought to build the Intelligent Transport System (ITS).

The Intelligent Transport System is way of creating local network and have the existing vehicles speak to each other and to other infrastructure with the help of various applications but as the network represents a solution it comes with other complications since having a digital identification opens the vehicles and the other infrastructures to networks attacks.

These network vulnerabilities rise the importance notion of security since it literally endangers the life of the human life which is the main reason of our project which tries to deal with the most dangerous type of attacks (Sybil attacks) as it holds a great impact to facilitate and even empower other types of attacks.

Many approaches were taken to deal with this attack using various protocols and authorities but with upcoming of machine learning it helped improve the results of many proposed solutions which is why machine learning is the method we have chosen to further improve the detection of Sybil attacks.

1. Chapter 1:
Project Context and Specifications

1.1. Introduction

We begin by presenting in detail the project theme and the problematic that we are working to solve.

1.2. Project Host

This work is part of a Franco-Tunisian research on the security vehicular networks, it is a partnership between the CReSTIC research laboratory of the University of Reims Champagne-Ardenne and the COSIM research laboratory of SUP'COM.

The STIC Research Centre (CReSTIC) is a research center as implied by its name which manages in various locations sites all imaging science research and innovation activities.

This research facility contains a 140 working people, 80 from them as research professors divided between two departments of “IT” and “Automation & Signal Processing” on 20 research teams to work especially in healthcare engineering, smart agriculture, 4.0 industry, creative industries, vehicle to vehicle communication and smart buildings

It is known to be a key player in innovation throughout the region with additional work on many national and international academic projects.

The COSIM laboratory Communication, Signals and Image Research Laboratory (COSIM) was inaugurated in 2011 under the supervision of the ministry of higher education and scientific research.

Containing approximately 60 researchers, COSIM is active in subjects concerning signal and image processing and numerical communication and its application on the national level.

1.3. Project Presentation

1.3.1. Scope

This internship project is elaborated in the context of a graduation requirement from the Higher school of communications of Tunis (Sup'Com) in the purpose of obtaining the engineering diploma.

The internship lasted for a period of six-months during which I integrated the Research group combined between the two laboratories to study the VANET security and various cyber-attacks and finally design and implement an artificial intelligence solution aiming to counter a designated type of cyber threat to the VANET.

1.3.2. General Concepts

1.3.2.1. Intelligent Transport System (ITS)

Intelligent Transport System (ITS) is a project of smart cities working as an advanced application aiming to provide innovative services revolutionizing the traffic management by sharing real-time running information.

This technology holds endless possibilities to provide numerous services other than traffic control like helping vehicles to be better informed of road safety ensuring a safer, more coordinated journey thanks to smarter transport networks.

Some of these technologies include calling for emergency services when an accident occurs, using cameras to enforce traffic laws or signs that mark speed limit changes depending on conditions.

The entire basis of ITS application is information sharing, analysis and processing cars which is the work of the Ad-hoc network between vehicles and rode side infrastructures under the name of VANET.

1.3.2.2. Vehicular Ad-Hoc Network(VANET)

1.3.2.2.1. Presentation of the VANET

Under the Intelligent Transport System (ITS) theme, evolution of the known mobile ad-hoc network (MANET), the Vehicular Ad-hoc network (VANET) is a Wi-Fi connection in a particular radius established between vehicles (V2V) and different infrastructure (V2I) with some environmental and characteristic exception.

The communication between vehicles is deployed in two manners: first using a direct connection with no infrastructure interference to relay messages quickly between cars hence its importance yet such connection is not permanent, secondly, the established interaction using infrastructure accommodates a more stable communication even if it is slow for the need of various services offered to many vehicles of the VANET.

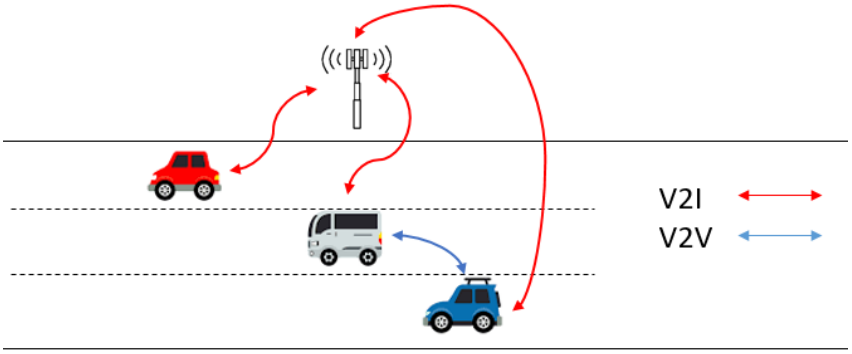


Figure 1: VANET connection possibilities

Thanks to the smart cities project, the progression of ITS is regarded as the improvement of its base(VANET). It is manifested by deploying various infrastructures, which diverge from the most needed Cellular BS to other RSU offering numerous services to reduce travel time and enhance comfort and security, such as the parking service and speed control.

The shared information is about the vehicle own properties such its position, speed and acceleration which can be detected thanks to numerous sensors built in modern vehicles such as radar and GPS for the position detection, these sensors are used to give the vehicle a detailed identity for itself and its surroundings [7].

VANET is a network for information sharing in between its nodes under strict protocols to ensure efficiency using three types of messages:

- RREQ (Route request): This type of message is used by a vehicle out or in a VANET to build or modify the topology of the VANET or to define a route to be taken by the vehicle.
- RREP (Route reply): This message comes in as a response to the previous RREQ message.
- RERR (Route error): This message is for when the link is cut so it is sent to the node source.

As an additional explanation of these messages' uses, we present some examples:

- Each node keeps track of its neighbors by listening hello messages that each node broadcast at defined interval.
- If a node wants to send data to the node which is not its neighbor, then it broadcast RREQ (Route Request) message.
- Every neighbor which receives RREQ, either return RREP (Route Reply) (if know the route to destination or destination itself) or rebroadcast RREQ.
- When destination is found, then RREP is sent via the reverse path.
- A higher sequence number signifies a fresh route, and the forward path is used to route data packets.

1.3.2.2.2. VANET architecture

The VANET architecture depends on the node's type: whether it is an infrastructure or a vehicle. A vehicle node of the VANET is made to be user friendly which explains the Application Unit (AU) and the On-board Unit (OBU) equipped inside the vehicle [3]:

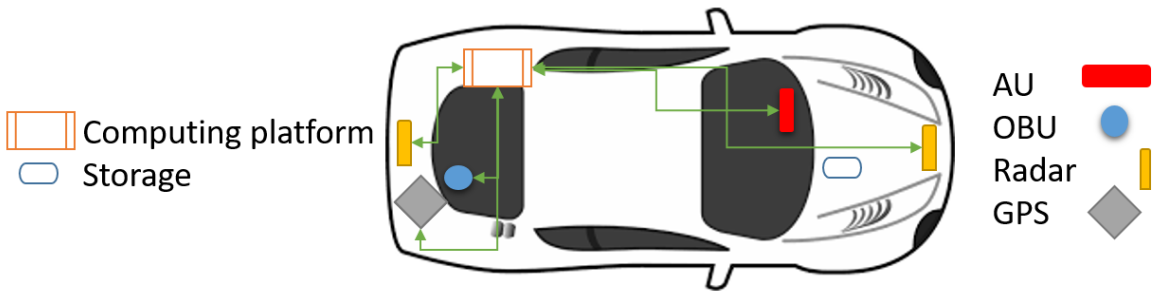


Figure 2:Vehicle node structure

- Application Unit: It can be an integrated part of a vehicle or a separate portable device (such as smartphone, laptop, etc..), used to run various applications and services provided by the device using the communication capabilities of an OBU in the VANET.
- On-Board unit: used to maintain connection directly or via multi-hop using Dedicated Short Range Communication (DSRC) [6] between OBUs of other vehicles and different infrastructure to insure sharing the information captured by the radar and GPS.

The infrastructure node uses:

- Road Side Unit (RSU): stationary device of an infrastructure to insure communication with other vehicles deployed on the side road to broadcast traffic information, route condition and weather condition as it may be used to connect between different nodes.

1.3.2.2.3. VANET characteristics

VANET is similar to the MANET yet largely differs because of the used end-devices in both networks. This vehicle Ad-hoc Network is bind with certain characteristics [13]:

- Highly variable mobility: the network is defined by the existing near vehicles and infrastructure and depending on the driver destination and behavior the topology of the built network will be easily modified making it harder to maintain a fixed topology for some tome like in the MANET.
- Traffic density: this value depends of the geographic area of the VANET, as with normal traffic the establishment of communications and exchange of information is possible until the point of high traffic area specially in urban areas where the nodes risks a storm of information.
- Provided energy source and storage: Unlike the MANET, the implemented equipment in vehicles to create the VANET does not pose an energy problem nor resources stocking thanks to vehicle's battery as energy source and an equipped storage device in the vehicle for information stocking

- Environment: the environment can be divided in two categories one in which the obstacles of the network such as trees and buildings are abundant

and the second when these obstacles does not cause an important problem like in the highway.

- Security: VANET offers the use of many applications to help the driver from which the application working to maintain the privacy and the security of information exchanged with messages of the VANET network.

1.3.2.2.4. VANET challenges:

The characteristics of the VANET imposes numerous challenges such the increase channel load when working in high density environment, let alone the frequent neighbor change but we are only interested on the notion of security and its complications in VANET.

In a VANET, each vehicle is associated with a digital identity which is the provided address identifying the vehicle to establish connection with other vehicles or with the infrastructures, unfortunately this new acquired digital identity and access to the network creates and leaves new attack surfaces on the open for a malignant person to exploit which directs us to the goal of this project: to design and implement an artificial intelligence solution to counter a designated cyber-attack.

1.3.2.3. Attacker models

Unfortunately, VANET is vulnerable to digital attacks hence compromising the privacy and security of drivers but the attacks can be consequence of many circumstances such is the type of the attacker:

- Active or Passive: A passive attacker can only eavesdrop on the transmission channel. This attack can be carried out by a curious neighbor, a company that seeks to create driver profiles. An active attacker can generate, modify, reject or replay messages to disseminate false information. The goal of active attackers is to grant themselves privileges to improve their driving environment like impersonating, for example, a rescue vehicle to facilitate its movement.

- Internal or External: An internal attacker is an authenticated member of the network who can communicate with the other nodes of the network since it

already has some advantages public keys used by other vehicles, for example. An attacker's internal attacker can cause more damage to the network than the external attacker who has limited access to the system.

- Malicious or Rational: A malicious attacker seeks to prove an ability or achieve personal success. To do this, it searches for vulnerabilities and exploits them to disrupt the system or injure members of the network. Attackers who deliberately cause road accidents are considered malicious. Therefore, the malicious attacker is ready to do anything to achieve its ends at any costs and consequences. By opposition, the rational attacker aims to accomplish a specific task on the network to the detriment (or in favor) of an identified person. Rational attackers are more predictable than malicious attacks.

- Ill-intentioned or Unintentional: An attacker is ill-intentioned if he purposely aims to challenge the proper functioning of the network. This model of the attacker should be distinguished from an unintentional attacker who can for example launch (unwittingly) an attack from a defective sensor.

- Independent or Collaborative: Attackers can act independently of each other or collaborate well. When they collaborate, attackers exchange messages and cooperate to make the attack more effective. For example, collaborative attacking vehicles announce a fictitious traffic jam to convince honest vehicles. These will then change the path, thus clearing the way for attackers.

- Local or Extended: An attacker has a limited range of action even if he controls several entities (OBU or RSU), as a result, a local attacker has limited scope. An extended attacker controls several scattered entities all over the network, which gives it a broad reach.

1.3.2.4. Attack examples

The existing threats are numerous which can be used to compromise the availability, authentication and confidentially of any node from the VANET from which these are presented [13]

- Denial of Service is an attack that occurs when the attacker takes control of a vehicle's resources and blocks the communication channel used by the Vehicle Network so that it prevents important and urgent information from reaching their destination like in the following figure the blue car is performing a type of denial of service attack by sending continuous messages to block the signal of the red car.

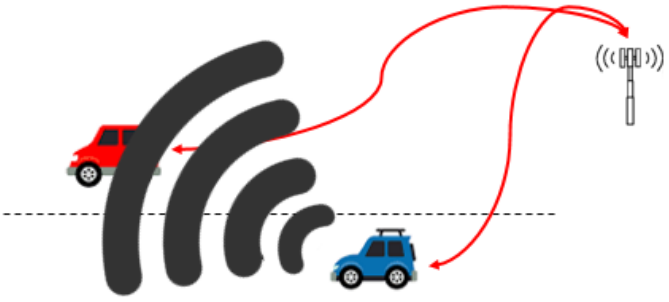


Figure 3: Example of denial of service attack

- Black hole attack is used by a node unwilling to redirect traffic as it must in the VANET and so it redirects received traffic to an inexistent node in the network or out of coverage causing data loss just like a black hole. In the figure 4 the attacking blue car performs the black hole attack by not redirecting the received message to the red car by redirecting the message a void digital identity presented by the crossed off circle.

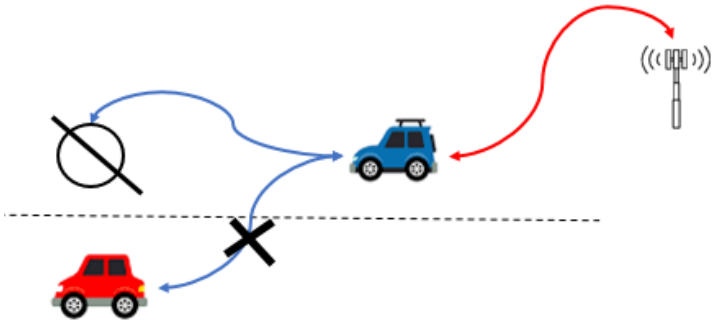


Figure 4: Example of black hole attack

- Malware attack: It is simply to inject a malware in the network and since there is a variety of malware and viruses that can be introduced to the network the most dangerous are as known as the zero-day malware representing a new undiscovered virus thus can't be countered. This attack can be performed on

any node of the network as well as the RSU depending on the level of security like the following figure.

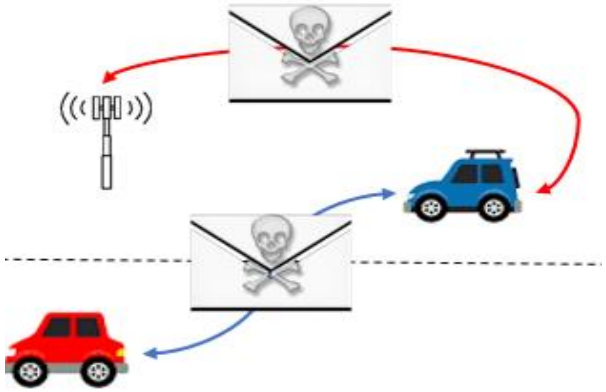


Figure 5: Example of Malware attack

- Attack on the coherence of information is when the malicious entity aims to inject and disseminate erroneous information on the state of road traffic in the network, to modify the behavior of other entities such as causing the route change of other vehicles or guiding them towards congestion, the compromised information can be cut off from an RSU signal or another vehicle to be modified.

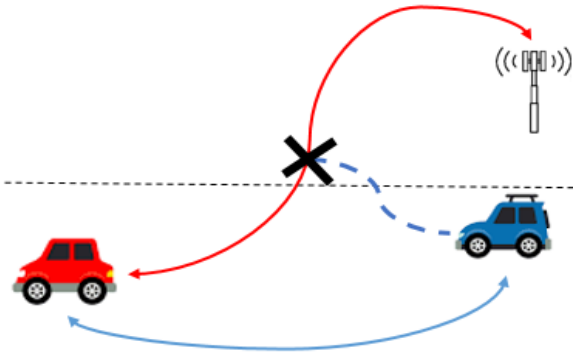


Figure 6: Example of information modification

- Deleting Messages: These packages may contain important information for the receiver. The attacker deletes these packages and can use them again at another time. Using such an attack to prevent the insurance authorities from knowing about accidents involving his vehicle and avoid issuing collision reports at roadside access points, the deleted information can be from an interrupted connection or a normal circulating communication that reached the attacker vehicle.

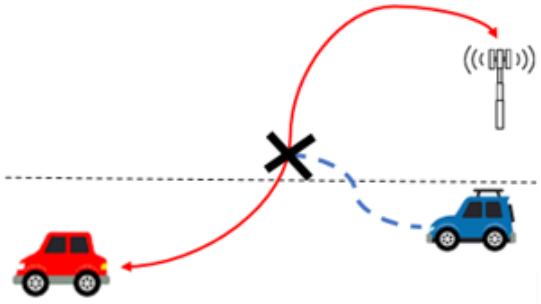


Figure 7: Example of message interruption and deletion

- Attack on privacy: The Privacy Attack is an attack that occurs when a malicious node tries to have the identity and personal information of another node in the network using eavesdropping especially since victim car will be sending GPS and destination information, to trace the activities and movements of the latter and might even identify the driver.

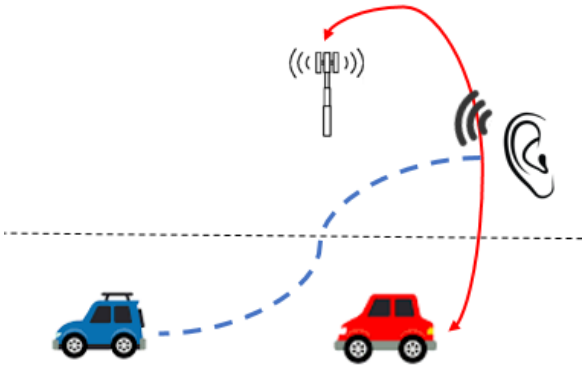


Figure 8: Example of eavesdropping

- Impersonation is similar to identity theft, the malicious node uses the identity of another node to be legitimate in the network and thus to have the functions of the latter, it is a technique called the spoofing of the identity which can be used on a node of the same VANET or out of coverage. In the next example the red car is inexistent but the blue car is masquerading as it to be able to send information under another identity

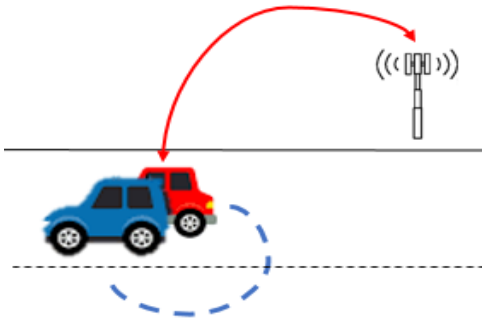


Figure 9: Example of impersonation

- Sybil attack: Sybil attack presented by Douceur [14] occurs when an attacker takes control of or creates numerous aliases to use them in various attacks for example: acts as if there are more vehicles on the road to inform other vehicles of false congestion and forces them to change the route, just like the blue car fabricating the existence of two nodes called Sybil nodes presented by the orange cars to attack the red car.

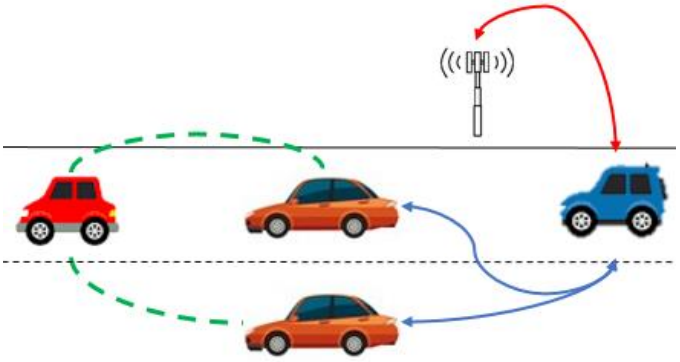


Figure 10: Example of a Sybil attack

1.4. Conclusion

In this part we laid down the context of the project talking about ITS and VANET leading to its challenges from which we are interested only by the cyber threats.

In a cyber-attack many entities are involved like (the driver, OBUs, RSUs, possibility of zombie node and the attacker) that is why we introduced the types of attackers in the VANET followed by an enumeration of possible types of cyber-attacks.

From the presented cyber-attacks, our goal is to deal with what can be considered the most dangerous cyber threat (Sybil attack) because it easily can be used to facilitates other cyber-attacks and even empower them for example: A DoS (denial of service) attack can use acquired Sybil nodes as zombies to perform a DDoS (Distributed Denial of Service) which holds same function but on larger scale thus wider impact.

2. Chapter 2:

State of the art

2.1. Introduction

For this chapter, we focus on presenting the Sybil attack and its impact accompanied with some techniques and methods proposed by different researchers

2.2. Sybil attack presentation

2.2.1. Sybil attack properties

In the VANET, each vehicle is presented as a single node of the network identified by a single digital identity and that is what a Sybil attack exploits as it is identified as a malicious node that fabricates numerous false identities to take control of the entire VANET and acquire the ability to inject falsified information into the network to cause damage to legitimate nodes.

During the Sybil attack there is two types of nodes:

- Sybil nodes: which are the ensemble of nodes created by spoofed identities of vehicles on the network or a distant one.
- Malicious nodes: the attacking node created by the Sybil node, it is the source of the attack.

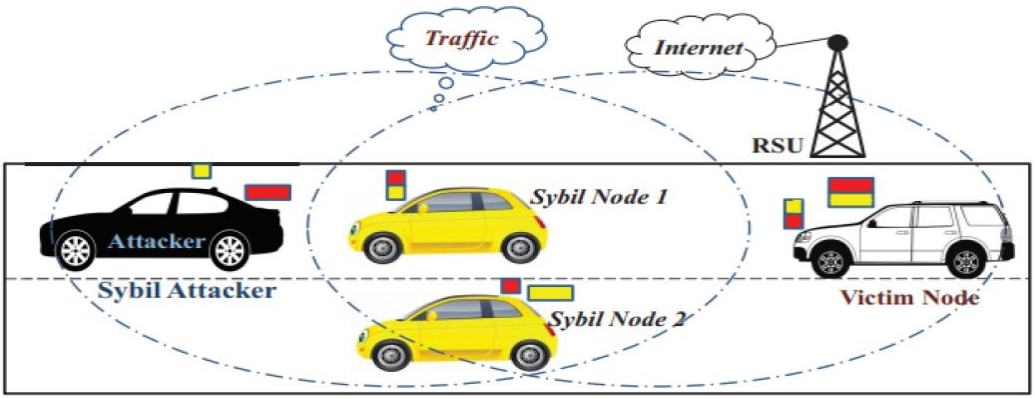


Figure 11: Sybil attack

It was first introduced by Douceur in peer-to-peer networks, the Sybil attack exploits these network vulnerabilities using many fake nodes under the attacker's or the group of attackers control on the ad-hoc network, creating an illusion about the existence of several vehicles in the network with identities that may be acquired by [15]:

- fabricating numerous fake IDs.
- spoofing the network's existing vehicles' IDs.
- spoofing an out of coverage vehicles' IDs.

After the manufacturing the Sybil nodes the established communication in between them in the VANET can be direct mostly in case of spoofing a legitimate physically existent vehicle thus communication can be limited between Sybil nodes and legitimate nodes, for the indirect type of communication it passes through the malicious node that fabricated the Sybil nodes mostly for the scenario of creating fake identities thus every messages that reached a Sybil node it continues to the malicious attacker and messages of the attacker are forwarded through Sybil nodes.

Thanks to Sybil nodes an attacker may pass to more brutal impact on network by exploiting Sybil nodes yet their participation depends between cases when they may be using them simultaneously or one at a time, even the used identity can be swapped in between multiple Sybil nodes or leave & join the network many times to confuse the network making it harder for detection.

2.2.2. Sybil attack effects:

Sybil attack has a large impact on Ad-hoc networks meaning VANET in our case, since it presents a multipurpose attack with countless consequences depending on the way to use it eventually this attack can lead to other complications in the network in the form of numerous types of intrusion of many objectives:

- To disrupt the generation of route in multi-itineraries or algorithms of geographical routing, by appearing in different places whilst the generation of routes like using signal jamming or DDOS attacks and can simply send false alert messages to clear the route and create a congestion in other places.
- Also in protocols of routing based on clusters, attacker Sybil is also able of disrupting the mechanism of head selection by confusing the voting system with the fabricated nodes. The system of vote is very useful to gather and prove certain precious information for various applications. Sybil nodes can modify the results of vote to falsify the report and identification of the node's poor behavior and the check on the vehicles position.
- Provocation of an innocent malicious behavior forced by the attacker actions mainly by using falsified messages or disrupting the services of legitimate car leading to a bad behavior from the legitimate car thus diminishing the trust of the legitimate node
- The node Sybil affects the resources allocation since it uses numerous identity facilitating their ability to hoard an unfair part of assets of the VANET.
- Can be used to disrupt the GPS of a legitimate car thus it possesses a falsified location.
- Keeping it for eavesdropping on as much as possible of existent nodes on the network…

To summarize all these attack models and more, it can all be classified to three main intrusion types:

- **Dropping packets**: An attacker may drop some chosen messages or an unlimited amount of messages depending of the needed attack by using different methods to be harder for detection thus more effective.

- **Insert wrong information or modify existing packets**: An attacker may create an illusion of a track jam before selecting an alternate route to its advantage by manufacturing a new message or modifying alert messages to create a certain topology and even redirect some vehicle to a designated road.
- **Replay and delay packets**: An attacker may fraudulently repeat or delay normal data transmission in VANET. This attack may seem useless but by repaying packets to other nodes it presents a case of impersonation of a legitimate node and for the delay, in critic situation like accidents in the road a little delay can lead to more chaos.

2.2.3. Detection techniques

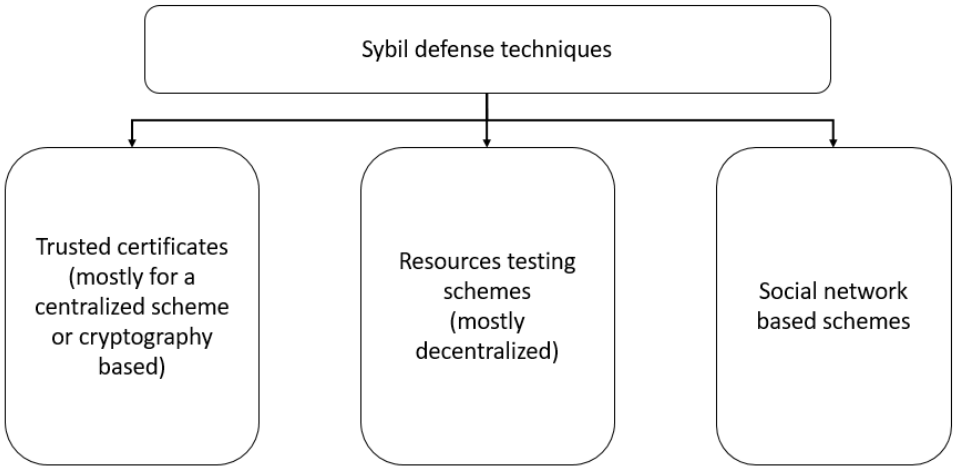


Figure 12: Categorization of Sybil defense techniques

The proposed categorization was according to various research that limited many types Sybil attack to those three outcomes.

The main goal of Sybil Attack detection is to ensure that each physical node is linked to a valid unique identity implicating that it is not fabricated, other approach is to verify and validate the messages circulating the VANET to ensure a more efficient security.

For this attack, various detection methods were designed and other protocols were created and implemented in many types of Ad-hoc networks to search the fastest and best results of Sybil nodes detection such as:

- Robust Sybil Attack Detection

- A cooperative Sybil attack in VANETs
- Distributed fusion algorithm to detect fake nodes
- PKI, digital certificates and time stamping
- Digital signatures
- Certification Authorities
- Cryptographic key distribution, certification and revocation are addressed

2.3. Related researches

For this part, out of the numerous developed researches on Sybil attack we will only be talking about the most recent researches concerning the VANET and using ML:

2.3.1. Proposed researches

a. Misbehavior Detection Model using Artificial Neural Network for Vehicular Ad hoc Network Applications

This project [2] uses a distributed model in between the nodes of the VANET: During this research an architecture was proposed divided their model construction on 4 phases:

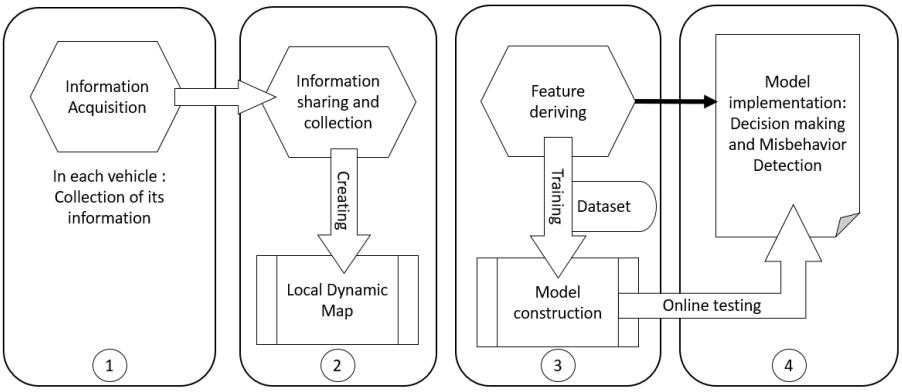


Figure 13: Proposed model construction phases (ANN in VANET)

First phase is for each vehicle to register its own information (speed, acceleration and direction) followed by information exchange between the VANET nodes using the robust acquisition algorithm to lastly formulate a local dynamic map by each node containing the received information.

This project explained the behavior of each vehicle (nodes of the VANET) by the derived local map which is used in the third phase for feature deriving then model construction using feed-forward backpropagation artificial neural network (ANN). The constructed model is implemented in the fourth phase to detect the unusual behavior of nodes which may be considered a Sybil node.

For this study is applied on Next Generation Simulation (NGSIM) dataset, which groups the vehicles under four clusters depending on the behavior (free-flow, lane change, flowing mode and random movement regime) using K-means clustering.

b. Collaborative Security Attack Detection in Software-Defined Vehicular Networks

This project [6] establishes a centralized connection: It presents an extension of their previous work where they proposed an architecture where the SDVC is divided into the control and data plane:

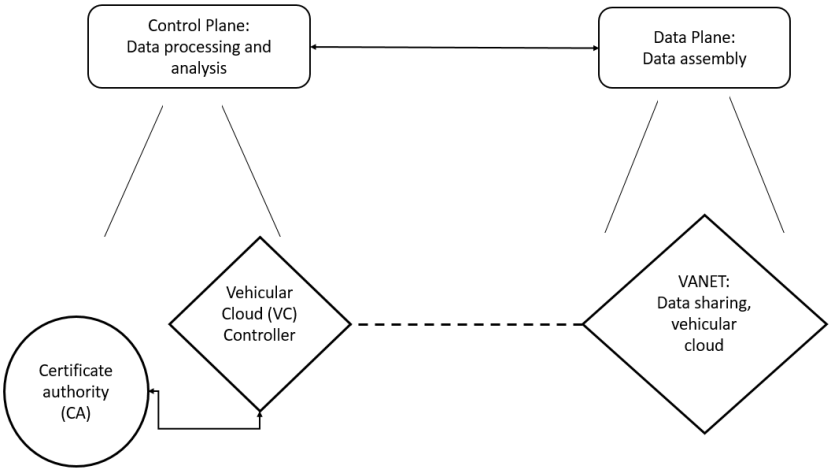


Figure 14: Proposed architecture of the SDVC

Each of these planes has an independent objective as the data plane accumulates the shared information in between the VANET nodes then send it to the control plane which is the combination between the vehicle cloud and the certificate authority.

Each vehicle is assigned a couple of private and public keys after entering the VANET by the certificate authority which they are used to sign sent messages to ensure their validity.

After capturing the received information from the Data plane, they are contained in the VC controller to train the multi-class SVM. The VC controller is provided with the needed resources of processing power and stocking devices since it is independent of the VANET's nodes, hence not limited.

Thanks to the assembled dataset of the vehicular cloud of the VANET, the VC controller has what is needed for the multi-class SVM training and testing.

The SVM machine learning technique is known for its large demanding of processing power and RAM storage which is dependent on the size of the dataset, fortunately, the use of an independent entity for computing, that can have its resources boosted, is applied for this project.

c. Robust Method for SAD

This project uses a centralized model in the RSU nodes of the VANET: It is based on differences in vehicle motion paths, assuming that people driving vehicles on their individual journeys maintain a certain speed and distance from other vehicle (nodes) for traffic or passenger safety.

In this approach [1] divided the steps to four main steps of Gathering, Calculation, Judging and Classifying.

First step begins when RSUs are provided with digital vehicle signatures as well as time stamps periodically or on request, to be recorded independently followed then by the next step to calculate the difference in between provided signatures and on third step use these distinctions to compare the signature vectors of the neighboring nodes to finish with classifying the similar signature as detected Sybil nodes.

In this technique, Digital signatures along with time stamps are broadcasted using only the infrastructure. As the vehicle enters the RSU range, the vehicle requests from the RSU an authorized message that proves the presence of a particular vehicle at the designated time in its range.

Authorized messages can be used to identify a vehicle location since the message depends on the current location with a certain timestamp, the collection of these messages allows the traveling vehicle to formulate its trajectory in succession with the corresponding time. The RSU compares in between trajectories to identify the vehicles with similar trajectory thus branding them as Sybil nodes.

Just like presented in the following table node3 & node4 having similar trajectory thus Sybil nodes:

Time	Node1	Node2	Node3	Node4 (Sybil node)
T_0	RSU3	RSU5	RSU5	RSU5
T_1	RSU4	RSU4	RSU4	RSU4
T_2	None	RSU3	RSU4	RSU4
T_3	RSU2	RSU2	RSU2	RSU2

Table 1: RSU messages of VANET nodes

d. A Sybil Attack Detection Scheme based on ADAS Sensors

This project uses a distributed model in between the nodes of the VANET: by exploiting the various sensors equipped to the modern vehicle including camera, radar, LIDAR (laser imaging detection and ranging), GPS (global positioning system) and other sensors such as microwave, infrared or ultrasonic radar.

Those sensors practically give eyes to the vehicle to help identify the surroundings and even the angle and distance of a detected object for that each fabricated node can have its physical existence verified to determine if it's a Sybil node.

In this approach [7], an assumption about fabrication of fake identities to create Sybil nodes having no physical existence thus exploiting these sensors to detect them. To boost this project accuracy an algorithm is proposed to capture multiple snapshot

(solution for no line of sight) and compare the acquired data from sensors by the ones from communication in VANET.

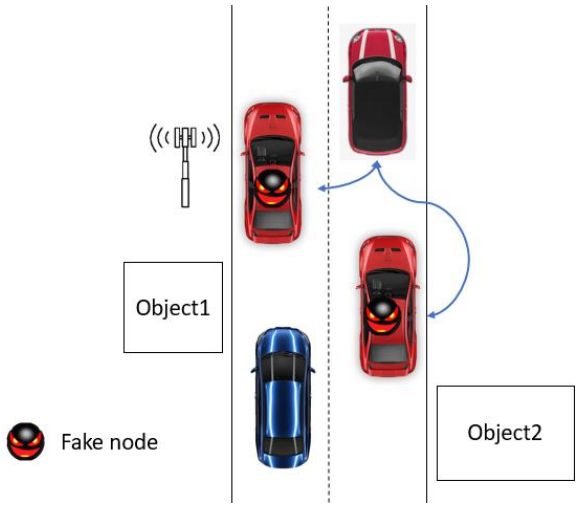


Figure 15: Example of a situation using this approach

Thanks to these sensors, the blue vehicle is able to identify surrounding red cars as Sybil nodes since they hold no physical existence and since this approach is based solely at the ability of sensors to detect surroundings, a machine learning algorithm such as YOLOv3 is chosen to increase the identification accuracy and the ability to defer between the surroundings objects.

This project presents interesting results but it limits its founding on a single scenario.

e. Misbehavior Authority System for Sybil Attack Detection in C-ITS

This project uses a distributed model as each node of the VANET detects the local potential misbehaving entities by checking the plausibility and consistency of some factors (speed, position ...) but centralized on the ML implementation.

The ITS-Station [5] is doing consecutive checks on the VANET for when a potential misbehavior is detected it sends a Misbehavior Report (MBR) to the Misbehavior Authority (MA) which is the central authority handling the misbehavior investigation.

The centralized investigation is divided in three consecutive phases each with its own goal:

- Detection of misbehavior's type in general: An implementation of multiple preprocessing technique followed by ML prediction using a recurrent neural network.

- Linking in between pseudonyms of the same vehicle: After the previous step it is essential to verify possible connection in between misbehavior detected pseudonym using after some preprocessing techniques one of two choices for the linkage in between pseudonym depending of the used architecture (IEEE SCMS for US architecture and a proposed neural network algorithm for the Europe architecture)
- Detection of Sybil attack type: Finally, using other preprocessing techniques on the dataset of results from previous step followed by Recurrent Neural Network.

For this approach, each step is dependent on the previous one since if no misbehavior is detected there is no need continue the process, and if no link is found between pseudonyms in the second phase the process is stopped before the beginning of the third step.

2.3.2. Comparison: proposed techniques of previous researches

Proposed techniques	Critics and limits of the research simulation and technique
Misbehavior Detection Model	This method under the assumption that every car has the needed resources for the computation of a model. The simulation uses for tests only four types of cars each with deferent traveling regime but depending on the types of environment there is problem of encountering additional behaviors thus limiting this proposed project.
Collaborative Security Attack Detection in SDVC	On of biggest problem of centralized approach is the congestion in the channel between the Vehicular Cloud and the VC controller since the dataset is collected from the information circulating in the VANET thus having a long waste of time before the processing and analysis of data for the model construction.

Robust Method for SAD	This works only under the assumption that an RSU is existent and not a victim of an attack. This research assumes each vehicle to maintain a fixed speed and distance for a better traffic safety which is not always the case of road traffic. The problem with having a Sybil node appear suddenly under the RSU with no trajectory to provide which can be using an obfuscated identity.
A Sybil Attack Detection Scheme based on ADAS Sensors	Talking about sensor detection which can easily deteriorate with longer distances between vehicle or on bad environments weather. This project based mostly on interactions using the provided sensors but not all vehicles has acquisition of all these sensors. Adding to that the limited use of a single scenario of Sybil attack.
Misbehavior Authority System for Sybil Attack Detection in C-ITS	Assuming the whole architecture works fine, this method can present a problem of the used ML linkage algorithm because of used protocol of communication. Some of the proposed simulation demanded more data because of its sensitivity to noise.

2.4. Conclusion

This chapter followed the presentation of Sybil attack (case of our study) and its effects on the VANET vulnerabilities which encouraged various researches to design useful techniques able to deal with this type of attack before causing a large impact on the VANET.

Such technique was dictated under the related research title followed by some critics and possible limitation concerning the technique and the simulation used in the research.

After this presentation, we continue to next chapter for the proposed method of this research as a detection technique of the Sybil attack.

3. Chapter 3: Proposed methodology and ML methods

3.1. Introduction:

In this chapter, we are detailing the used method on the provided Dataset, then some preprocessing techniques to finally talk about the chosen ML methods for this project.

3.2. Proposed preprocessing techniques:

Preprocessing is an important step before the use of the ML methods because whatever is the dataset, the preparation made before the ML interaction holds great importance on the results.

3.2.1. The Matrix of movement

This approach is used on numerous research in 2017 [4] then in 2020 [8] which proved its interesting results on large dataset. It is a preprocessing method used to reduce and define the behavior of each vehicle by a matrix noting its behavior (Driving Pattern Matrix) noting for each line of the matrix is consecutive registered information of the vehicle:

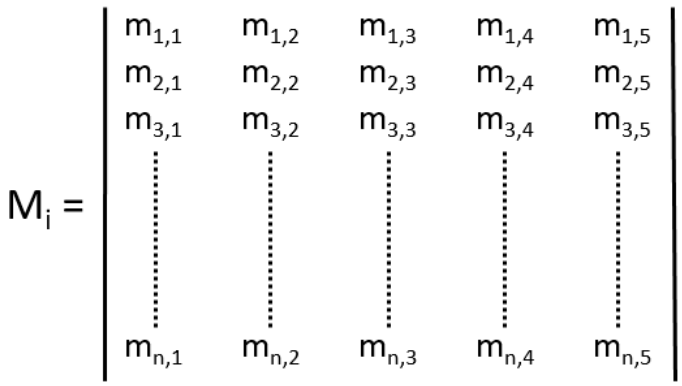


Figure 16: Behavior matrix presentation

To create this Matrix, certain features must be chosen to present the behavior of the vehicle such these five features (timestamp, the position of the vehicle, the speed of the vehicle, the acceleration and the variation of the vehicle's acceleration) to create a vector at a certain point of time t which corresponds to the vector of movement $V_{i,,t} =(m_{t,1},m_{t,2},m_{t,3},m_{t,4},m_{t,5})$. The consecutive register of the vector of movement and their compilation in ascending order of timestamps is what creates the matrix M, for this matrix a variation of provided features are chosen during this research to improve results.

For the matrix A: lambda is the corresponding eigenvalue to the eigenvector x:

{ A * x = lambda * x } and the square matrix can be presented by diagonalization into a matrix:

$$\begin{bmatrix} LAMBDA2 & 0 & 0 \\ 0 & LAMBDA1 & 0 \\ 0 & 0 & LAMBDA \end{bmatrix}$$

Figure 17: Form of a diagonal matrix 3x3

A Matrix has eigenvectors and eigenvalues to explain the distribution of the output of variable from the Matrix. The reason of this method is that each eigenvalue explains how the dataset is spread out on the corresponding eigenvector and eigenvalues. The most dominant eigenvectors/eigenvalue couple containing the most of the dataset of the matrix are the largest eigenvalues.

After organizing these matrices an algorithm aiming to reduce the dimension of the matrix by replacing each matrix by its two highest eigenvalues to limit errors as much as possible, hence these chosen eigenvalues represent the behavior of the corresponding vehicle.

Thanks to this method, for each pseudonym presenting a certain vehicle node a set of two eigenvalues is chosen, hence reducing the lines of dataset composed of the whole assembled messages to only some lines of different pseudonyms and their eigenvalues.

This is the used algorithm for calculation and choosing of the eigenvalues developed using the python language:

Algorithm: computing Eigenvalues function

Require: Matrix of Movement M
Ensure: Two chosen eigenvalues
1. nligne ← number of rows in m
2. ncolon ← number of columns in m
3. for i ← 1 to ncolon do
4. $mean_i$ =sum of elements in the $column_i$ /nligne
5. end for
6. al ← 0
7. ac ← 0
8. while al < nligne do
9. while ac < ncolon do
10. $m_{al,ac} \leftarrow m_{al,ac} - mean_{ac}$
11. al ← al+1
12. ac ← ac+1
13. end while
14. end while
15. Msquare ← transposeM*M
16. eigenvalueslist ← eigenvalues(Msquare)
17. for j in eigenvalueslist do
18. $eigenvalueslist_j$ ← absolute value of $eigenvalueslist_j$
19. end for
20. sort in reverse eigenvalueslist
21. return first two values of eigenvalueslist

```
1  def eig_val(mat):
2      ligne_mat=mat.shape[0]
3      column_mat=mat.shape[1]
4      list_mean=mat.mean(0)
5      ac=0
6      al=0
7      while al < ligne_mat:
8          while ac < column_mat:
9              mat[al,ac]-=list_mean[ac]
10             ac+=1
11         al+=1
12     mat1=np.dot(mat.transpose(),mat)
13     w_EV=eig(mat1)[0].tolist()
14     w_EV1=[]
15     for i in w_EV:
16         w_EV1.append(abs(i))
17     w_EV1.sort(reverse=True)
18     return [w_EV1[0],w_EV1[1]]
```

Figure 18: Proposed algorithm to calculate eigenvalues

First the two line 1 and 2 for taking the number of matrix lines and columns respectively followed by three lines for the registration of the mean of each column in a list which is used from line 8 to 14 by reducing each values of the column of the matrix with its corresponding mean as a centralization method proving to help avoid the generation of complex numbers. To calculate the eigenvalues of a matrix it needed to be square matrix explaining the used function in line 15 to multiply the matrix to its

own transpose and finally line 16 for the computation of the list of eigenvalues which is sorted in descending order in the following lines to return in the last line the first two eigenvalues of the list thus presenting a summary of the used matrix.

As a result, from a dataset holding numerous values and several features we are down to two features proving the importance of such methods to reduce complexity in training.

3.2.2. Modeling techniques

While using an unbalanced dataset it is useful to lean on modeling techniques to fix the biased form of the dataset which are represented by:

- Under sampling: takes up the dominant class in the number of lines from the dataset and reduces it to the same number of rows as the second class.
- Oversampling: opposite to under sampling, it increases the number of rows of the label having least corresponding lines to match the second label holding larger number of lines of the dataset.

Now to present the chosen sampling techniques of techniques:

3.2.2.1. Under sampling

3.2.2.1.1. Random Under-Sampling

Random subsampling involves randomly selecting examples from the majority class and deleting them from the training dataset. In random subsampling, majority class instances are rejected at random until a more balanced distribution is achieved.

3.2.2.1.2. Tomek Links for Under sampling

It is a subsampling method developed by Tomek. It is considered an improvement of the nearest neighbor rule (NNR). The algorithm works as follows: Either x an instance of class A and y an instance of class B. Either d (x, y) the distance between x and y. (x, y) is a T-Link, if for an instance z, d (x, y) d (x, z) or d (x, y) d (y, z) If two examples are T-Link, one of these examples is a noise or otherwise the two examples are located at the limit of the Classes. The T-Link method can be used as a guided subsampling method where majority class observations are deleted.

3.2.2.1.3. Edited Nearest Neighbors Rule for Under sampling

Another rule for finding ambiguous and noisy examples in a data set is called Edited Nearest Neighbors, or sometimes ENN for short. This rule involves using k = 3 closest neighbors to locate examples in a dataset that are misclassified and then deleted before a k = 1 classification rule is applied. This approach to re-sampling and classification was proposed by Dennis Wilson in his 1972 article entitled “Asymptotic Properties of Nearest Neighborhood Rules Using Edited Data”.

3.2.2.2. Combined sampling

This is simply using both techniques of random oversampling and random under sampling:

Random oversampling is a technique that involves randomly duplicating examples from the minority class and adding them to the training dataset. This combination between the use of oversampling on the minority class and under sampling on the larger class can prove useful even if the two methods did not improve the results when were used alone.

3.3. Machine learning methods

Machine Learning is a scientific field, and more specifically a subcategory of artificial intelligence which involves letting algorithms discover recurring patterns, in datasets. These data can be numbers, words, images, statistics…

Anything that can be stored digitally can be used as data for Machine Learning. By detecting patterns in these data, algorithms learn and improve their performance in the execution of a specific task since machine Learning algorithms learn autonomously from the training dataset and improve their performance over time. Once trained, the algorithm tries to detect patterns in the test data.

3.3.1. XGBoost

Gradient Boosting is a supervised learning algorithm using the Model aggregation method whose principle is to combine the results of a set of simpler and weaker models in order to provide a better prediction by assigning weights to each classifying model and used data

The used classified models are used on classification and using the assigned weights the results are eventually combined into a single result

3.3.2. ELM

ELM (Extreme Learning Machines) are feedforward neural networks. “Invented” in 2006 by G. Huang.

ELM uses the neural network to classify the given data, and to prepare the neural network the input weights are assigned randomly but It uses Moore-Penrose generalized inverse to set the output weights, meaning that it computes the output weights with the inverse matrix.

- **H** is called **Hidden Layer Output Matrix**
- T is a training data target matrix

$$T = H\beta \quad <=> \quad \hat{\beta} = H^{\dagger}T$$

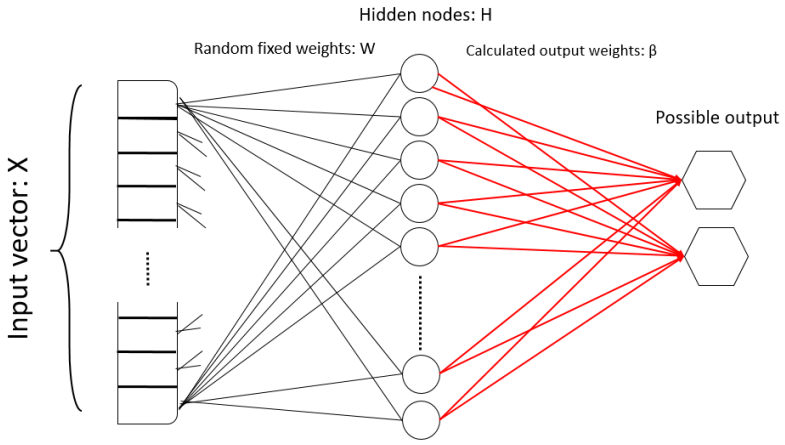


Figure 19: ELM hidden network (scenario of 2 outputs)

3.3.3. AdaBoost

An AdaBoost classifier is a meta-estimator that begins by fitting a classifier on the original dataset and then fits additional copies of the classifier on the same dataset but where the weights of incorrectly classified instances are adjusted such that subsequent classifiers focus more on difficult cases.

It is similar to XGBoost but without the weights on models.

3.3.4. KNN

K nearest neighbors uses the calculated distance to arrange the dataset next to each other until it has groups of data each corresponding to a class.

It is considered a type of instance-based learning or non-generalizing learning as it is not about creating a general model but to try and store entities from the training data in the corresponding group afterwards for each entities of the testing data we only need to compute the distance and figure out the corresponding class.

3.3.5. Decision Tree

It is about creating a tree until it reaches the assigned depth or the leaf node. It learns to partition on the basis of the attribute value. It partitions the tree in recursively manner call recursive partitioning to create simple decision rules inferred from the data features.

3.3.6. Random forest

A random forest is another meta estimator that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting.

Normally the best type of split is when using the whole features or a random max subset of them which is useful to decrease the variance of the forest estimator by introducing the randomness.

3.4. Scoring methods:

In this project, we are working with two classes only defined as 0 for innocent vehicle/node and 1 for the malignant one that is why we can assign the positive criteria to the first class (0) and the negative criteria to the second class (1).

3.4.1. Confusion Matrix

A Confusion Matrix is a summary and representation of the results of predictions in a binary classification is a 2×2 matrix about a classification problem. True and False

predictions are highlighted and broken down by class and the results are thus compared with the actual values.

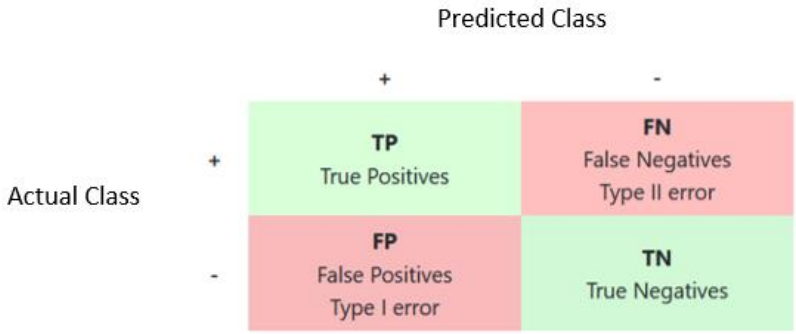


Figure 20: Confusion matrix (In case of only two classes)

These matrices are useful in statistics, data mining, machine learning models and other applications of artificial intelligence.

3.4.2. Accuracy

It is the most used metric to evaluate the ML method, thanks to it being easy to calculate, easy to interpret, and is a single number to summarize the model's capability.

IT is defined as the fraction of validly assigned label from the entire entities:

$$accuracy = \frac{TP + TN}{TP + FN + TN + FP}$$

However, accuracy fails to examine the performance for an imbalanced classification problem, where the distribution of examples in the training dataset across the classes is not equal. Because achieving 90% classification accuracy may be trivial on an imbalanced classification problem. This means that accuracy misleads us into thinking that a model has good or even excellent performance when it does not. In such case, we resort to the use of F1-score metric to evaluate the model.

.

3.4.3. Precision

This metric verifies only one of the classes according to the predicted labels.

It is defined as the rate of the validly assigned label of a certain class from the entire entities of the same class according to the predicting model. In the case of using two classes, the precision corresponds to rate of the positive class:

$$precision = \frac{TP}{TP + FP}$$

3.4.4. Recall

This metric verifies only one of the originally assigned classes.

Similar to precision it tests only one class and it is defined by the number of relevant labels found in relation to the number of relevant labels in the original database. In the case of using two classes, the precision corresponds to rate of the positive class:

$$recall = \frac{TP}{TP + FN}$$

3.4.5. F1-score

The F1 score is a binary classification metric that takes into account both the recall and the precision of binary metrics. This is a weighted average of Precision and Recall, it is the harmonic mean between the sensitivity and the predicted positive rate:

$$\text{F1-score} \qquad \frac{2 * precision * recall}{precision + recall}$$

The range is between 0 and 1 provided that the larger the value is, the better the predictive accuracy.

3.4.6. ROC curve

The ROC curve is a graphical representation of the existing relationship between the sensitivity and specificity of a test for all possible threshold values. The ordinate (y-axis) represents the sensitivity and the abscissa (x-axis) corresponds to the quantity (1 - specificity).

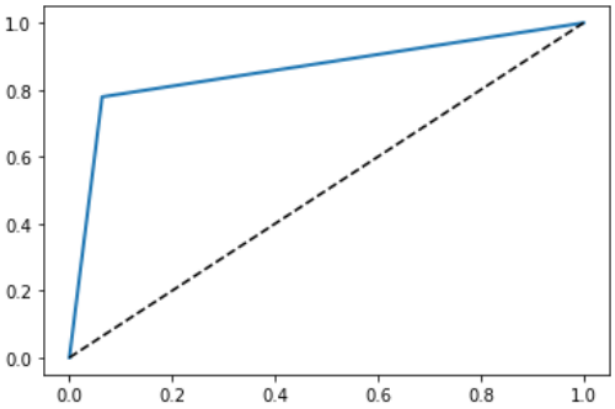


Figure 21: Example of ROC curve

3.5. Conclusion

This chapter began by the presentation of the preprocessing method implemented in this research (Matrix of movement) and its advantage of reducing the complexity of ML model creation followed by some interesting techniques of sampling that were needed because of the unbalanced dataset of VeReMi used in this research.

In addition, a series of ML algorithms implemented in the research to be able to choose from them the most compatible with the provided dataset which is essential since results can be critically variable depending on their compatibility. Many algorithms were used but only the most promising algorithm were presented.

Finally, a detailed presentation of the implemented scoring methods during this research to defer between interesting approaches and failures. Some of the scoring methods are useful more than others in this case of study, still the use of many scoring methods helps to insure the identification of the best possible outcome.

4. Chapter 4:

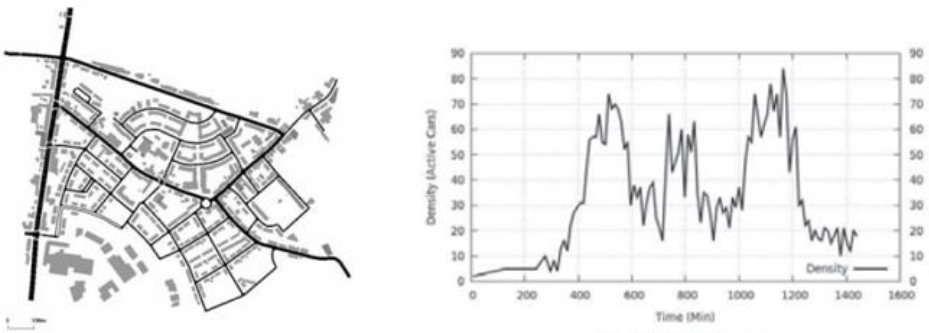
Offline results

4.1. Introduction

In this chapter, we will be introducing the dataset and the results of the proposed solution

4.2. VeReMi dataset presentation:

4.2.1. Simulation Platform



(a) Network: Luxembourg City (b) Vehicle Density

Figure 22: Simulation Scenario

The Vehicular Reference Misbehavior (VeReMi) dataset [10] as implied by the name an assembly of information denoting the evaluation of misbehavior detection mechanisms for the VANETs. VeReMi is a simulated dataset, generated using LuST (Version 2) which is an open source synthetic traffic scenario validated with real data provided by the Vehicular Lab of the University of Luxembourg and VEINS an open source simulator for Inter-Vehicular Communication is based on OMNeT++ and SUMO

(with modifications, based on Version 4.6). OMNeT++ is a C++ simulation library for building network simulators and SUMO is a well-known open source road traffic simulation suite presented in the Figure 21.

The VeReMi dataset contains a large load of separated data each corresponding to different attack from which we are using the Sybil attack dataset. This dataset is a collection of message logs from vehicle's on-board units, it includes a file labelled ground truth. The dataset contains malicious messages used to trigger an incorrect application behavior, which is what the misbehavior detection based solutions aims to prevent.

The collected log messages with help of DSRC communications with vehicles' node contains both the GPS data under the type 2 and BSM labeled as type=3 [16].

4.2.2. VeReMi dataset: Feature study

4.2.2.1. Existent features

For each vehicle, a temporary ID is provided during the VANET established at first connection to the network which is represented as the 'senderPseudo' in the dataset.

In the dataset we have also the ID of each node under the sender feature.

For each message [11]:

- An Id of the message (messageID) is used
- The type of the log message
- The timestamp of the log message.
- The position of the Vehicle defined by a coordinate of the vehicle on the plan using the meter scale
- The velocity of the vehicle on both vectors of the coordinate on the meter per seconds scale.
- The heading of the vehicle on both vectors of the coordinate

Accordingly, a sensor error models was used on the four main data fields: Position, Velocity, Acceleration and Heading [10].

Position Error: positioning systems vary on the levels of precision which also differ by region, the Global Positioning System (GPS) precision is limited to about 3 to 5 m in an open environment and up to 20 m in urban areas proving the need of an internal correction system on-board every vehicle built in backed by the position error model as follows:

P_t	$\triangleq$	Real Position at time t
$P_t^{\mathcal{E}}$	$\triangleq$	Broadcasted Position at time t
$\mathcal{E}_t^P$	$\triangleq$	Position Error at time t
U	$\triangleq$	Uniform Distribution
$\mathcal{N}$	$\triangleq$	Normal Distribution

$$\mathcal{E}_0^P = U([-5, 5])$$
$$\mu = \frac{\mathcal{E}_0^P + \mathcal{E}_{t-1}^P}{2} \qquad \sigma = 0.03\mathcal{E}_0^P$$
$$\mathcal{E}_t^P = \mathcal{N}(\mu, \sigma^2)$$
$$P_t^{\mathcal{E}} = P_t + \mathcal{E}_t^P$$

Velocity Error: Using the wheel spin with an average error around 0.05m/s the vehicles estimate velocity, since the error is also proportional to the velocity, the speed error model is as follows:

V_t	$\triangleq$	Real Velocity at time t
$V_t^{\mathcal{E}}$	$\triangleq$	Broadcasted Velocity at time t
$\mathcal{E}_t^V$	$\triangleq$	Velocity Error at time t

$$\mu = 0 \qquad \sigma = 0.00016$$
$$\mathcal{E}_0^V = \mathcal{N}(\mu, \sigma^2)$$
$$V_t^{\mathcal{E}} = V_t + V_t * \mathcal{E}_0^V$$

Acceleration Error: As a conclusion from the velocity error the acceleration error model is as follows:

A_t	$\triangleq$	Real Acceleration at time t
$A_t^{\mathcal{E}}$	$\triangleq$	Broadcasted Acceleration at time t

$$A_t^{\mathcal{E}} = A_t + \frac{\mathcal{E}_t^V - \mathcal{E}_{t-1}^V}{\delta t}$$

Heading Error: A vehicle's heading may be calculated using a magnetic compass or calculated from successive positions. But The accuracy of a magnetic compass is dependent on the device quality as the research of H¨olzl et al. found that the probability of having an error below 20◦ is around 85% for most mobile devices, however Deelertpaiboon et al. managed to successfully reach an accuracy of 0.1using the magnetic compass on a vehicle.

Additionally, accuracy of the heading computed from successive positions is dependent on the vehicle velocity since at a high velocity the position heading is more probable to have higher accuracy than the compass heading, whereas the opposite is

true at lower speeds or when a vehicle is stationary thus the following heading error model was used:

H_t	$\triangleq$	Real Heading at time t
$H_t^{\mathcal{E}}$	$\triangleq$	Broadcasted Heading at time t

$$\mathcal{E}_0^H = U([-20, 20])$$
$$\mathcal{E}_t^H = \mathcal{E}_0^H * e^{-0.1 * V_t}$$
$$H_t^{\mathcal{E}} = H_t + \mathcal{E}_t^H$$

Finally, all the features of the VeReMi dataset can be presented by this list in correspondence with the domain of possible values.

```
{"type":ℤ[0,20],
 "rcvTime":ℝ[0,+∞],
 "sendTime":ℝ[0,+∞],
 "sender":ℤ[0,+∞],
 "senderPseudo":ℤ[0,+∞],
 "messageID":ℤ[0,+∞],
 "pos":[ℝ[−∞,+∞],ℝ[−∞,+∞],ℝ[−∞,+∞]],
 "pos_noise":[ℝ[0,+∞],ℝ[0,+∞],ℝ[0,+∞]],
 "spd":[ℝ[−∞,+∞],ℝ[−∞,+∞],ℝ[−∞,+∞]],
 "spd_noise":[ℝ[0,+∞],ℝ[0,+∞],ℝ[0,+∞]],
 "acl":[ℝ[−∞,+∞],ℝ[−∞,+∞],ℝ[−∞,+∞]],
 "acl_noise":[ℝ[0,+∞],ℝ[0,+∞],ℝ[0,+∞]],
 "hed":[ℝ[−∞,+∞],ℝ[−∞,+∞],ℝ[−∞,+∞]],
 "hed_noise":[ℝ[0,+∞],ℝ[0,+∞],ℝ[0,+∞]]}
```

Figure 23: The features of VeReMi's Sybil attack dataset

As presented by this figure the position, position noise, speed, speed noise, acceleration, acceleration noise, heading and heading noise features are all presented by a list of three values each correspondent to one axis of the three dimensional space: [X, Y, Z].

4.2.2.2. Added features

Before the begging of any our work on the dataset a simple division is applied on the features presented by list of three values to create two more columns for each feature (on for the X axis, the second for the Y axis) to make the access to these values easier.

4.2.2.2.1. Prediction of the position

First feature to be added is the prediction of the vehicle's position in the designated time which is to be computed from the existent attribute of the dataset:

To calculate the predicted position, the dataset is sorted by the sender Pseudo ID on the first level and in ascendant time by sorting it on second level using timestamps afterwards we use this function on every line but since we are using old message the first predicted position of each vehicle is given the first position by default:

Algorithm: computing predicted position function

Require: difference in time: T, current speed: S, current position: P, current acceleration: A
Ensure: the predicted position after the difference in time
1. predicted_position ← P+T*(S+T*A)
2. return predicted_position

```
def predicted_position(delta_time,speed,old_pos,accel):
    f=old_pos+delta_time*(speed+delta_time*accel)
    return f
```

Figure 24: Algorithm for position prediction

- delta_time: difference in time between the old message and the current one
- speed: the declared value of the old message
- old_pos: the declared value of the old message
- accel: the declared value of the old message

This function is implemented on the first two coordinates [X, Y] to return a list of the two coordinates' predicted position in this form: ['predict_loc_x','predict_loc_y']

4.2.2.2.2. Acceleration variation

Second Added feature was proposed by the research from which we inspired this work which takes interest in the acceleration's variations of the vehicle between consecutive sent messages thus it returns a list of these values in the form of two coordinates ['delta_acl_x','delta_acl_y'] corresponding each message.

This added feature was used to compare its corresponding research with this proposed work method: the found results proved poor results with the used dataset VeReMi in comparison to the used method.

4.2.2.2.3. Predicted position plausibility:

For this feature, it is to be added as another column (feature) after the computation of the eigenvalues of each matrix and containing all of them in a single data frame which is to be used for the model creation.

Algorithm: assign label depending on predicted position function

```
Require: Matrix of movement M
Ensure: list of labels on two axis ( X , Y )
1.  Sort the Matrix by ascending time
2.  nli ← number of M rows
3.  for j ← 1 to nli do
4.      if difference between predicted position and current position on the X axis > 50 then
5.          labelX ← 1
6.      else  labelX ← 0
7.      end if
8.      if difference between predicted position and current position on the Y axis > 50 then
9.          labelY ← 1
10.     else  labelY ← 0
11.     end if
12.     add (labelX,labelY) to labellist
13. end for
14. return labellist
```

Figure 25: Algorithm for plausibility factor of the original dataset

This algorithm assigns the plausibility factor that is derived out of the difference between vehicle's predicted location and current position: each predicted position with a large difference of the declared position in the log message was given the value of 1 on the corresponding coordinate otherwise the value 0 is given.

As written in the algorithm, the chosen difference is strictly bigger than 50m whether on the X coordinate or the Y coordinate.

Algorithm: assign average of the labels from the predicted position function

Require: dataset, list of the senderIDs in the dataset:Lsender
Ensure: list of labels of one element(0 or 1)
1. for sender in Lsender do:
2. select from dataset the rows of sender only
3. meanX ← calculate the average of prediction labels on X axis
4. meanY ← calculate the average of prediction labels on Y axis
5. meanlabel ← (meanX+meanY)/ 2
6. if meanlabel > 0 then
7. meanlabel ← 1
8. end if
9. add meanlabel to labellist
10. end for
11. return labellist

```
1  def sum_pred_label(data,sender):
2      list_label_pred=[]
3      for sendr in sender:
4          data_micro=data[data['senderPseudo']==sendr].sort_values(['sendTime'],ignore_index=True)
5          alpha1=mean(data_micro['pred_label_1'])
6          alpha2=mean(data_micro['pred_label_2'])
7          alpha=(alpha1+alpha2)/2
8          if alpha>0:
9              alpha=1
10         list_label_pred.append(alpha)
11     return list_label_pred
```

Data: used dataset
Sender: list of senderpseudo
Alpha1: average of X axis of the plausibility factor
Alpha2: average of Y axis of the plausibility factor

Figure 26: Algorithm for average plausibility calculation

After assigning such values, we calculate the mean of the predicted plausibility column concerning each sender Pseudo and on each axis thus acquiring ‘alpha1’ and ‘alpha2’ which are used to get their average ‘alpha’. If the resulted value is different than 0 then the plausibility value assigned to the sender Pseudo is 1 otherwise the value 0 is given.

Finally, in line 10 the average plausibility factor of each ‘senderPseudo’ is added in a list which is assigned in the dataset containing eigenvalues.

This chosen feature has an impact on label prediction as it seems like a simple technique of class prediction which eventually resulted in improving the model construction.

4.3. ML offline results

The use of the Behavior matrix wastes a considerable time to process the eigenvalues that is why we chose two kinds of lists of features to work with:

- First list=['sendTime','pos_x','pos_y','predict_loc_x','predict_loc_y']

- Second_list=['sendTime','pos_x','pos_y','spd_x','spd_y','acl_x','acl_y','hed_x','hed_y','delta_acl_x','delta_acl_y']

The first list was chosen after many trials to decide the most impactful features.

The second list is proposed by the research to compare in between both lists' result.

4.3.1. Implementation of the Machine learning method

4.3.1.1. First Step: Choosing ML method

First step was to search for the most coherent method with the provided dataset based on the results of F1-scores and the confusion matrix results.

For that we used only the first list with no type of modeling to identify the most convenient ML method to be used with our Dataset.

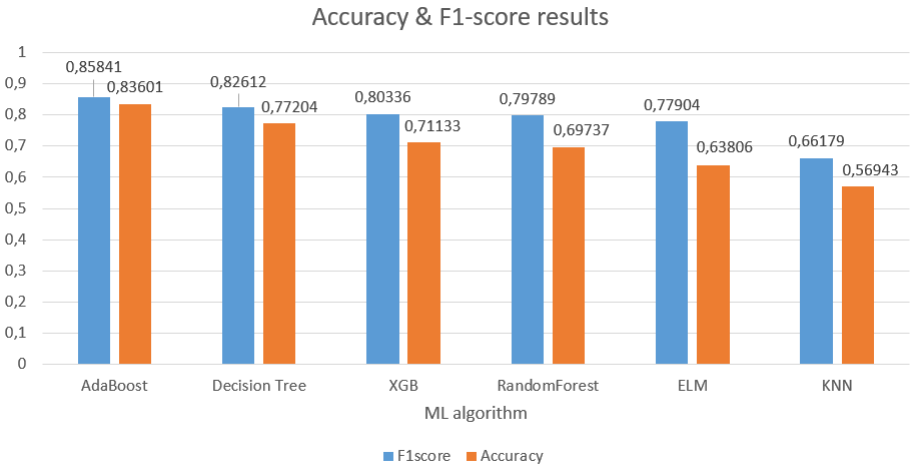


Figure 27: Histogram of F1-score & Accuracy using different ML algorithm

Confusion Matrix: AdaBoost	Predicted label 0	Predicted label 1
Actual label 0	94,6%	6,4%
Actual label 1	22%	78%

Confusion Matrix: Decision Tree	Predicted label 0	Predicted label 1
Actual label 0	64%	36%
Actual label 1	15%	85%

Confusion Matrix: XGB	Predicted label 0	Predicted label 1
Actual label 0	34%	66%
Actual label 1	7,6%	92,4%

Confusion Matrix: KNN	Predicted label 0	Predicted label 1
Actual label 0	41%	59%
Actual label 1	34%	66%

Confusion Matrix: Random Forest	Predicted label 0	Predicted label 1
Actual label 0	28%	72%
Actual label 1	6,4%	94%

Confusion Matrix: ELM	Predicted label 0	Predicted label 1
Actual label 0	0%	100%
Actual label 1	0%	100%

Figure 28: The confusion matrices of each ML algorithm in percent

Out of these results, obviously AdaBoost worked remarkably with our dataset and out of the rest method we can only tolerate both the Decision Tree and XGBoost results

since we did not use any modeling techniques so there is a possible window for improvement.

⇨ We chose these techniques (AdaBoost, Decision Tree, XGBoost) to continue our work.

4.3.1.2. Second Step: Comparing in between lists

For this part we simply conducted the same processing on dataset but with different chosen features (both declared lists: First and Second) to create the behavior matrix and compute their corresponding eigenvalues.

In this step without using modeling techniques, both the created dataset of eigenvalues is applied to the three chosen ML methods which leads to this compilation of F1-score results:

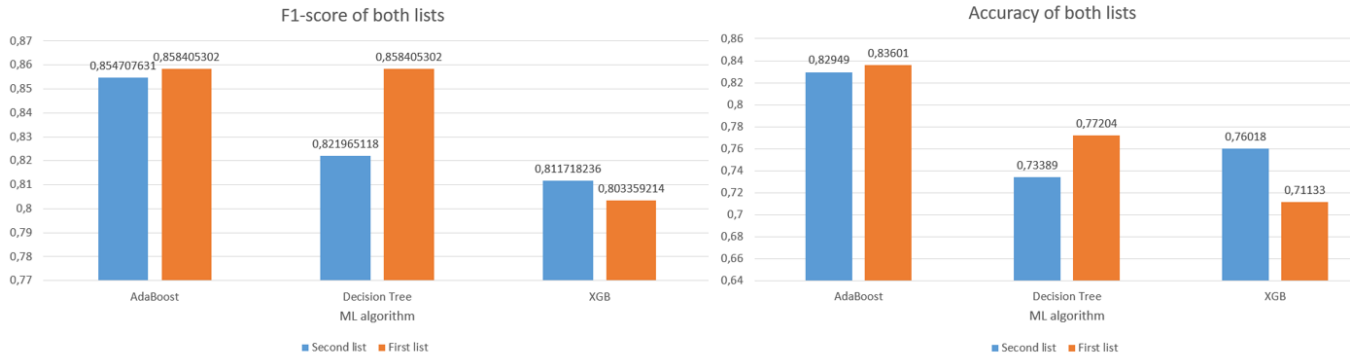


Figure 29: Accuracy and F1-score comparison between both lists

Even if the results are close to each other, out of this outcome, the derived conclusion is:

- The first list is more useful to create the model than the research proposed one
- The AdaBoost ML method proven its importance as the best of the chosen ML methods.

After comparing the F1-score we use the confusion matrix to verify our conclusion:

Confusion Matrix: AdaBoost	Predicted label 0	Predicted label 1
Actual label 0	94,6%	6,4%
Actual label 1	22%	78%

Confusion Matrix: Decision Tree	Predicted label 0	Predicted label 1
Actual label 0	64%	36%
Actual label 1	15%	85%

Confusion Matrix: XGB	Predicted label 0	Predicted label 1
Actual label 0	34%	66%
Actual label 1	7,6%	92,4%

Figure 30: First list confusion matrices

Confusion Matrix: AdaBoost	Predicted label 0	Predicted label 1
Actual label 0	91%	9%
Actual label 1	21%	79%

Confusion Matrix: Decision Tree	Predicted label 0	Predicted label 1
Actual label 0	57%%	43%
Actual label 1	13%	87%

Confusion Matrix: XGB	Predicted label 0	Predicted label 1
Actual label 0	44%	56%
Actual label 1	10%	90%

Figure 31: Second List confusion matrices

4.3.1.3. Third step: Improving the results

For this task, we continued with only the first list since it proved being more promising in the previous step. Now to observe the modeling techniques effects we tried many variants of which mostly for under sampling.

The conducted experiment is establishing this modeling techniques on the training data only then continue to run the AdaBoost ML method for each case explaining the following results:

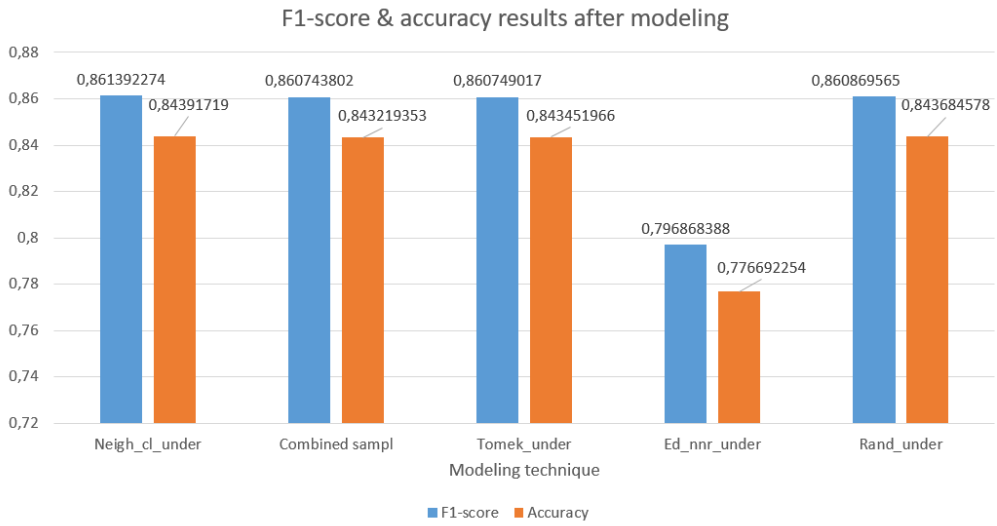


Figure 32: comparison between modeling techniques

Except for the edited nearest neighbor under sampling technique, all the observed confusion matrices were all practically the same with as follows:

Confusion Matrix: AdaBoost	Predicted label 0	Predicted label 1
Actual label 0	99%	1%
Actual label 1	24%	76%

Figure 33: Confusion matrix of most modeling techniques

⇨ Out of these techniques we only saw big difference with only the edited nearest neighbor under sampling technique which is to be omitted because it is not convenient to our case of study.

Another choice for results improving was relying on the third added feature: Predicted position plausibility.

As it is explained before this feature is eventually added to the eigenvalues Matrix taking values of 1 or 0 only and this is the observed outcome:

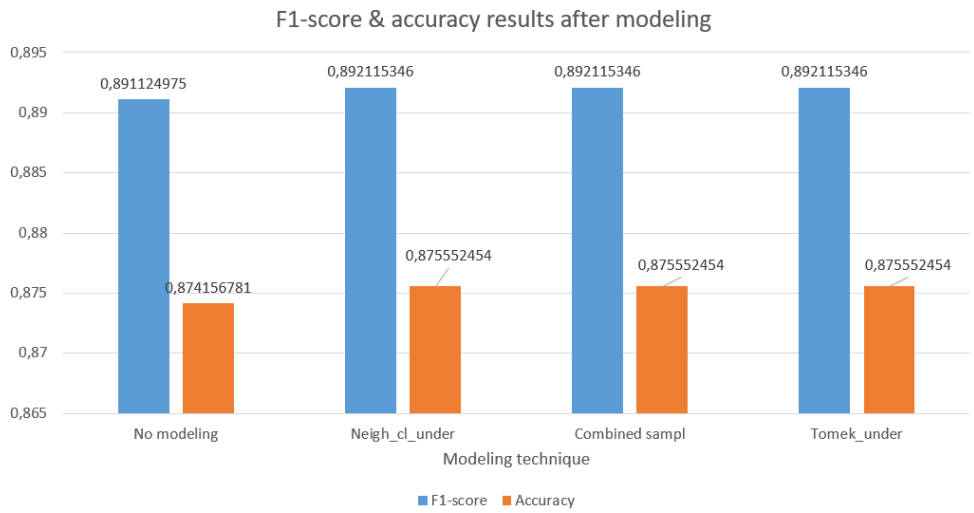


Figure 34:results after adding the plausibility feature

It is obvious that these results seem to be very close with variation of maximum 0.001 that is why we turned to the confusion matrix but found that they round up to the same conclusion with little variation:

Confusion Matrix: AdaBoost	Predicted label 0	Predicted label 1
Actual label 0	99,23%	0,77%
Actual label 1	19%	81%

No modeling

Confusion Matrix: AdaBoost	Predicted label 0	Predicted label 1
Actual label 0	99,74%	0,26%
Actual label 1	19%	81%

neighbor clean under sampling

Confusion Matrix: AdaBoost	Predicted label 0	Predicted label 1
Actual label 0	99,74%	0,26%
Actual label 1	19%	81%

Combined sample

Confusion Matrix: AdaBoost	Predicted label 0	Predicted label 1
Actual label 0	99,74%	0,26%
Actual label 1	19%	81%

Tomek under sampling

Figure 35: Confusion matrices of modeling techniques

⇨ Finally, out of these results we can safely assume that we reached the best preprocessing steps thus the most valuably convenient ML model out of the provided Dataset which can be explained by the evolution of ROC curve to this point from deciding of the ML method to be used to how improve the results:

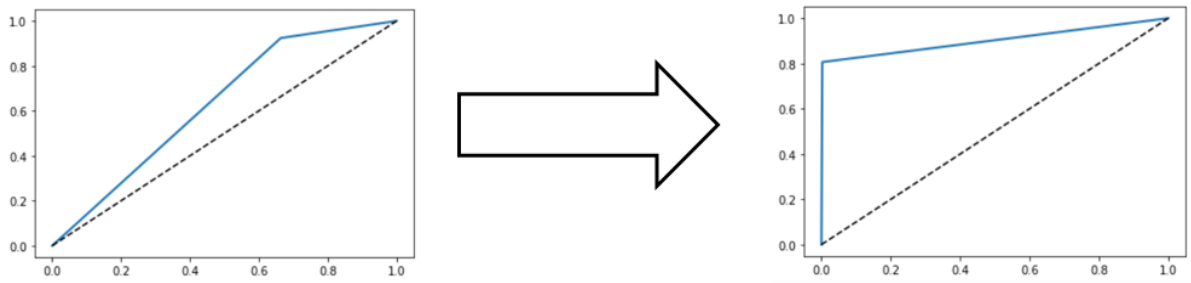


Figure 36: Evolution of ROC curve

4.3.2. Comparison: Brut dataset & dataset with preprocessing

This part is to observe the improvement provided by the proposed method in comparison to the use of the whole dataset in their brut form.

4.3.2.1. Results comparison: F1-score and confusion matrix

As usual we begin with a histogram of F1-score:

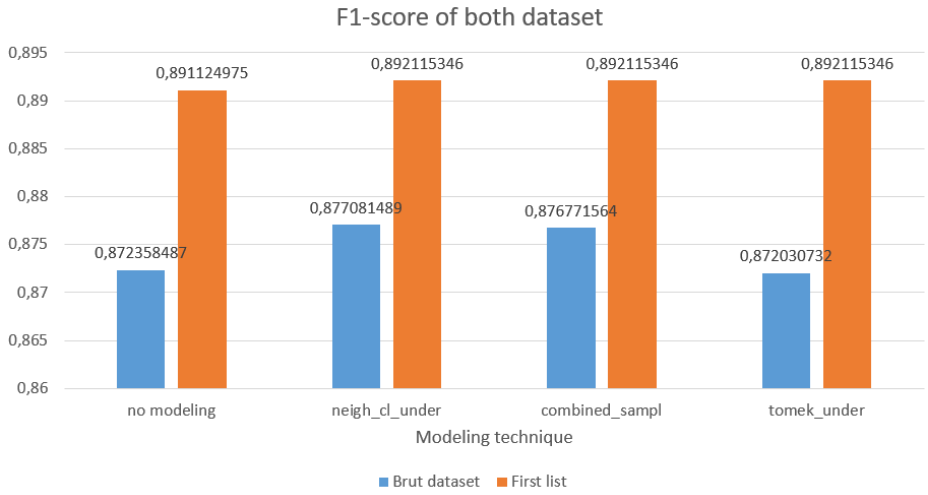


Figure 37: Comparison between both datasets

This results of comparison between the created dataset and the provided dataset are proven even after using modeling techniques with a large difference. We can easily comment on the difference between the F1-score results of 0.02 which is important since it F1-score is the harmonic rate between precision and recall and such difference presents interesting consequences on one or both of them.

Thus using the confusion matrix, we can observe such difference:

Confusion Matrix: AdaBoost	Predicted label 0	Predicted label 1
Actual label 0	99,79%	0,21%
Actual label 1	22%	78%

Brut Dataset

Confusion Matrix: AdaBoost	Predicted label 0	Predicted label 1
Actual label 0	99,23%	0,77%
Actual label 1	19%	81%

Created dataset from first list

Figure 38: Comparison between confusion matrices

We can remark the percentage of true negatives having big difference since the brut dataset works with large lines of data (447615) thus the percentage of false negative is too great comparing to a data lines of (4299).

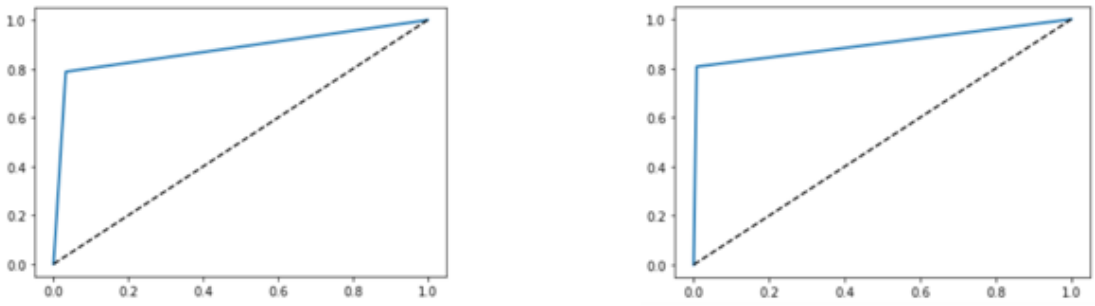


Figure 39: ROC curves of both datasets

4.3.2.2. Advantages of proposed method

- The most interesting advantage is the ability of the matrix method to identify the used senderpseudo as a Sybil node which in contrast to using the unrefined method (implementing the ML methods on the whole dataset) will only identify the detected messages as malignant.
- The major waste of time while computing the ML method:

Average of work time	Brut dataset	Refined dataset
AdaBoost (no modeling)	3min+20sec	0,92sec
AdaBoost (neigh_cl_under)	27min+29sec	0.94sec
AdaBoost (tomek_under)	26min+20sec	0.92sec
AdaBoost (combined_sampl)	3min+39sec	1sec

Table 2:Comparison between used time for model construction, training and prediction

4.3.2.3. Disadvantages of proposed method

The biggest disadvantage about this method is the time of work to form the matrix and calculate the eigenvalues of behavior reaching average of 8min but this processing time can be dealt with a higher performing device than my machine.

4.4. Conclusion

In this part we talked about the used dataset and its features then continued to present our work design and implementation by specifying first the chosen features to

be added to our dataset then we began by detailing the results of this part of the research, the difference and improvement that was made by the proposed method.

Finally, we can assume that the proposed methods presented the best possible results in comparison to the use of the unrefined dataset and the proposed selection of features (Second list) from another research paper.

Sure that this method still holds some shortfalls but a window of improvement can always be opened to find better outcome.

5. Chapter 5: Online Results

5.1. Introduction

For this chapter, we begin first by a presentation for the simulation environment to be used next an application of code modification to improve the used detection solution. This simulation environment is the source of our VeReMi dataset and is an accumulation of different application which are to be detailed.

5.2. Setting up the environment

5.2.1. Application structure

An accumulation of three different applications can be explained by the next figure:

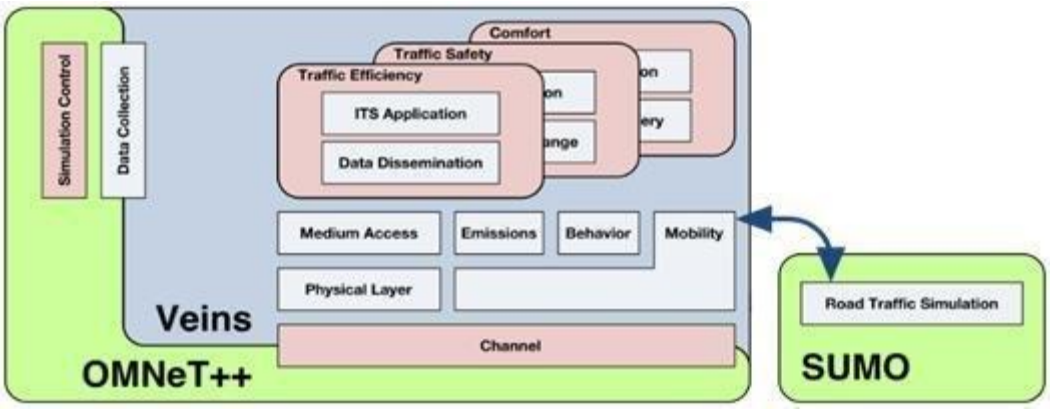


Figure 40: Relation between SUMO, OMNeT++ and VEINS

Simulation of Urban MObility (SUMO) [17] is an open source application offers modelling of intermodal traffic systems including road vehicles, public transport and pedestrians, which simply means that SUMO presents the graphic simulation of vehicles transportation on a certain road map.

Since Its development, which was mainly by employees of the Institute of Transportation Systems at the German Aerospace Center, numerous supporting tools were developed to facilitate some core tasks such as network import, route calculations, visualization and emission calculation. Thanks to being open source, it can still be enhanced using custom models.

OMNeT++ is another open source simulator but for the network interactions and communications. It is an extensible, modular, component-based C++ simulation library and framework used for a graphical presentation of simulating wired and wireless communication networks, on-chip networks, queueing networks, and so on.

OMNeT++ provides a component architecture for models. Components (modules) are programmed in C++, then assembled into larger compound component using a high-level language (NED). OMNeT++ has extensive GUI support, and due to its modular architecture, the simulation kernel (and models) can be embedded easily into your applications.

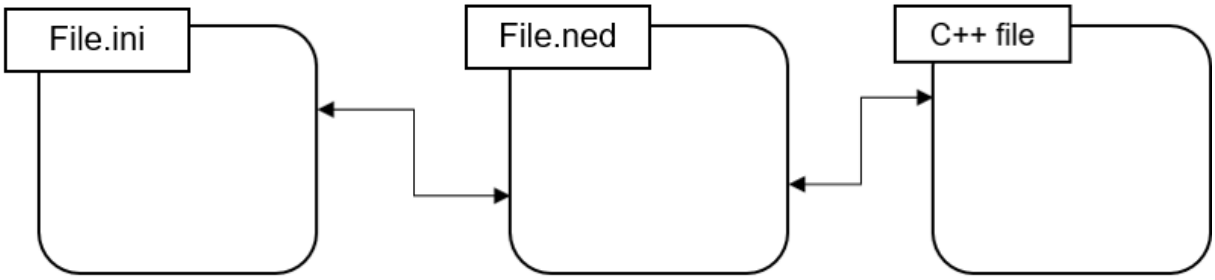


Figure 41: Example of OMNeT++ scenario

In simpler terms, an OMNeT project uses the File.ini as the configuration file where all simulation parameters are accumulated which points to the Ned (Network description) file that presents the used modules and submodules on the simulated network.

The behavior of the nodes on the network is simulated by the used class (defined as module in OMNeT) with all its parameters and the methods developed using the c++ code. The used class is declared using “@class (used class name)” in the Ned file. Three methods are always existing in the used class:

- initialize(): invoked upon launching the simulation on to initialize all used parameters in the simulation followed by launching the coded behavior
- finish(): this method is for when the simulation is stopped
- handleMessage(cMessage *msg): called when a node gets message to follow the coded task.

Such models can be reused as the OMNeT team maintain lists of certain models just like the framework INET which offers models for Internet stack and many other protocols and components. This INET framework used as base for the vehicular networks (Veins).

Veins is an open source vehicular network simulation network. It holds a large audience of users as it is still under active development. It uses the two previous application to create to stable simulation of vehicles on a chosen road map. The Veins framework provides numerous models to work with on the VANET simulation such as Framework For Misbehavior Detection (F²MD) which is used in this case of study.

F²MD is an application offering a division the work on the network simulation in three servers which is presented by the attacker server, misbehavior detection server and offering the use of a machine learning server to help the performance of misbehavior detection on the active simulation. The results are displayed using numerous graphic plots.

F²MD proposes another behavior to work with its new architecture, this behavior is noted in the numerous declared and developed classes using C++. All these classes are called following the main class named “JosephVeinsApp” (work of Kamel Joseph) which uses in some cases as basis the original “MyVeinsApp”.

5.2.2. Preparing the environment

We begin by choosing the OS (linux ubuntu in this case), then creating a virtual box image with certain characteristics:

- CPU: 3.
- Storage: minimum of 30 Go.

- Ram: 6 Go.
- Ubuntu 20.04.2 LTS.

then to downloading the needed libraries and resources using this command:



Figure 42: SUMO needed dependencies

Next is the SUMO installation which can be easily by using:

"sudo apt-get install sumo sumo-tools sumo-doc"

which easier but won't provide the latest version, or to go with the manual installation which begins with the download the latest version of sumo from this site: https://sourceforge.net/projects/sumo/files/sumo/ and extract the downloaded document:

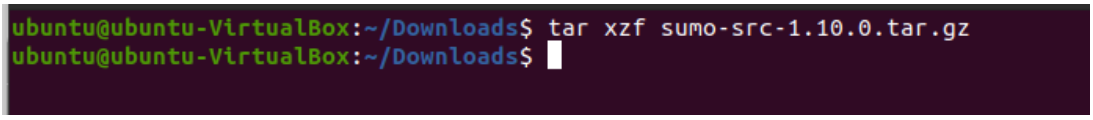


Figure 43: SUMO package extraction

Next for the installation of sumo, we create this repository "cmake-build" using followed by the activation of installation program:



Figure 44: The used commands for SUMO installation

Now finally we add to the ".bashrc" file the Path to the sumo environment (bin file of sumo) using the gedit command for more flexibility to sumo use:

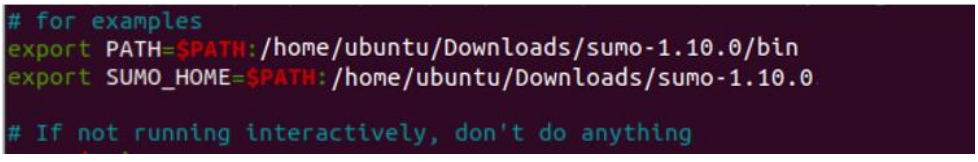


Figure 45: Environment variable of sumo in bash file

A modification to the sumo file names is needed to have synchronization in between used application (from sumo-guiD to sumo-gui, and from sumoD to sumo)

```
ubuntu@ubuntu-VirtualBox:~/Downloads/sumo-1.10.0/bin$ mv sumo-guiD sumo-gui
ubuntu@ubuntu-VirtualBox:~/Downloads/sumo-1.10.0/bin$ mv sumoD sumo
```

Figure 46: SUMO files name change

Next is OMNeT++ installation, as always we begin by downloading the needed libraries and dependencies:

```
ubuntu@ubuntu-VirtualBox:~$ sudo apt-get install build-essential gcc g++ bison flex perl python python3 qt5-default libqt5opengl5-dev tcl-dev
tk-dev libxml2-dev zlib1g-dev default-jre doxygen graphviz
ubuntu@ubuntu-VirtualBox:~$ sudo apt install openscenegraph osgearth
ubuntu@ubuntu-VirtualBox:~$ sudo add-apt-repository ppa:ubuntugis/ppa
ubuntu@ubuntu-VirtualBox:~$ sudo apt-get install openscenegraph-plugin-osgearth libosgearth-dev openmpi-bin libopenmpi-dev libpcap-dev nemiver
Reading package lists... Done
Building dependency tree
Reading state information... Done
```

Figure 47: Installing dependencies for OMNeT++

It is needed to change the file variable kernel.yama.ptrace_scope in “/proc/sys/kernel/yama/ptrace_scope”, to the value of 0 from 1. Then continue to package download and extraction:

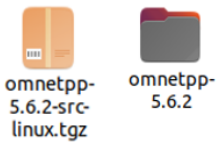


Figure 48: OMNeT files after extraction

After downloading preparing the OMNeT++ application, it is time to set the environment variable which refers to setting the path on the “.bashrc” by running the “. setenv” command followed by the installation using “./configure” and “make” program:

```
ubuntu@ubuntu-VirtualBox:~/omnetpp-5.6.2$ . setenv
ubuntu@ubuntu-VirtualBox:~/omnetpp-5.6.2$ ./configure
ubuntu@ubuntu-VirtualBox:~/omnetpp-5.6.2$ make
```

Figure 49: Setting the environment then installing OMNeT++

OMNeT++ is now accessible using the “omnetpp” command.

For the F²MD app preparation: after installing the git command to use Github, this process may take a large time so to insure the work in synergy between OMNeT++ and F²MD they must be under the same Path (/home/ubuntu in our case)

We clone the Github repository of joseph Kamel concerning the F²MD:

```
ubuntu@ubuntu-VirtualBox:~$ git clone --recurse-submodules https://github.com/josephkamel/F2MD.git
```

Figure 50: Github cloning of F²MD repository

Then proceed to the installation:

```
ubuntu@ubuntu-VirtualBox:~/F2MD$ ./buildF2MD
```

Figure 51: F²MD installation command

Finally, the VANET simulator is prepared and in the following step we try a sample scenario using some simple modifications.

5.2.3. Sample presentation

The veins framework offers five kinds of sample scenarios to try them such a zone in Paris (plateau de Paris-Saclay) under the name of IRTXScenario and Luxembourg scenario using three types of road size.

5.2.3.1. Chosen scenario

The chosen scenario is IRTXScenario. First a change in omnetpp.ini file must be made to have the misbehavior detection server of F²MD interact with the ongoing simulation, as for the attacking parameters we won't be using the server but using the omnetpp.ini we can control the chosen attack and the number of attacker to finally build our VeReMi dataset:

```
ubuntu@ubuntu-VirtualBox:~$ cd F2MD/veins-f2md/f2md-networks/IRTSystemXScenario/
ubuntu@ubuntu-VirtualBox:~/F2MD/veins-f2md/f2md-networks/IRTSystemXScenario$ ls
antenna.xml       gui-settings.cfg.xml      IRTSystemX_bbox.osm.xml  IRTSystemX.polycfg      omnetpp.ini
config.xml        IRTSystemX.0.rou.alt.xml  IRTSystemX.launchd.xml   IRTSystemX.poly.xml     results
coordinates.txt   IRTSystemX.0.rou.xml      IRTSystemX.netccfg       IRTSystemXScenario.ned  run
debug             IRTSystemX.1.rou.alt.xml  IRTSystemX.net.xml       IRTSystemX.sumo.cfg     trips.trips.xml
decals.xml        IRTSystemX.1.rou.xml      IRTSystemX.png           memcheck                type.add.xml
ubuntu@ubuntu-VirtualBox:~/F2MD/veins-f2md/f2md-networks/IRTSystemXScenario$
```

Figure 52: Scenario of Paris-Saclay files

```
ubuntu@ubuntu-VirtualBox:~/F2MD/veins-f2md/f2md-networks/IRTSystemXScenario$ gedit omnetpp.ini
```

Figure 53: Editing the omentp.ini file

Changing the parameter "*.node[*].appl.sendReportsV1" to "true" activates the use of misbehavior detection server offered by F²MD module

```
#################
# Reporting Parameters
#################
# Report Versions ... 0:OneMessageReport, 1:EvidenceReport, 2:ProtocolReport,
3:BasicCheckReport
*.node[*].appl.appl.REPORT_VERSION = 2
*.node[*].appl.sendReportsV1 = true
*.node[*].appl.sendReportsV2 = false
*.node[*].appl.maHostV1 = "localhost"
*.node[*].appl.maHostV2 = "localhost"
*.node[*].appl.maPortV1 = 9980
*.node[*].appl.maPortV2 = 9981
#################
```

Figure 54: Reporting parameters of omnetp.ini file

For the attack server, since we have no interest to other attacks we won't be activating the server but we will assign to the parameter "*.node[*].appl.LOCAL_ATTACK_TYPE" the number 16 referring to the attack GridSybil and we can vary this parameter "*.node[*].appl.LOCAL_ATTACKER_PROB" to assign the number of attacker in the simulation which in this case 0.5.

```
#################
# Attacks Parameters
#################
*.node[*].appl.UseAttacksServer = false
*.node[*].appl.MixLocalAttacks = false
*.node[*].appl.RandomLocalMix = false
*.node[*].appl.LOCAL_ATTACKER_PROB = 0.5
# Local Attack Types ... 0:Genuine, 1:ConstPos, 2:ConstPosOffset, 3:RandomPos,
4:RandomPosOffset, 5:ConstSpeed
# Local Attack Types ... 6:ConstSpeedOffset, 7:RandomSpeed, 8:RandomSpeedOffset,
9:EventualStop, 10:Disruptive,
# Local Attack Types ... 11:DataReplay, 12:StaleMessages, 13:DoS, 14:DoSRandom,
15:DoSDisruptive, 16:GridSybil,
# Local Attack Types ... 17:DataReplaySybil, 18:DoSRandomSybil, 19:DoSDisruptiveSybil
*.node[*].appl.LOCAL_ATTACK_TYPE = 16
*.node[*].appl.GLOBAL_ATTACKER_PROB = 0.0
# Global Attack Types ... 20:MAStress
*.node[*].appl.GLOBAL_ATTACK_TYPE = 20
#################
```

Figure 55: Attacking parameters of omnetp.ini file

Now after saving this configuration, we begin the simulation by launching the Traci module:

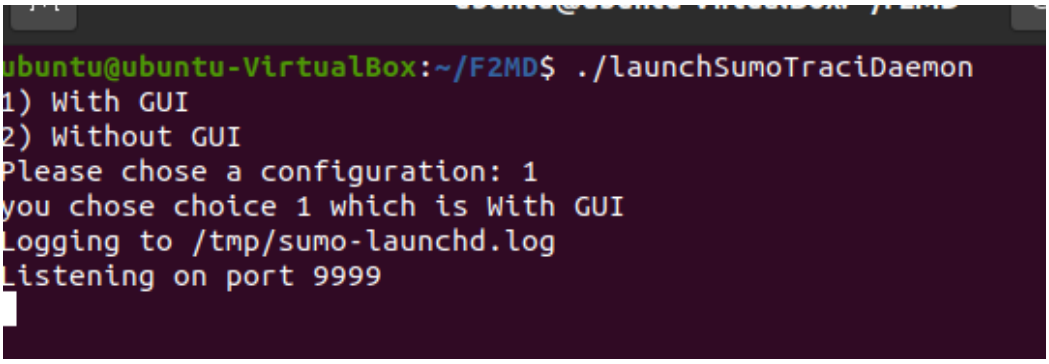


Figure 56: Launching the Traci Daemon bash program

Afterwards the misbehavior detection server:



Figure 57: Launching the misbehavior detection server bash program

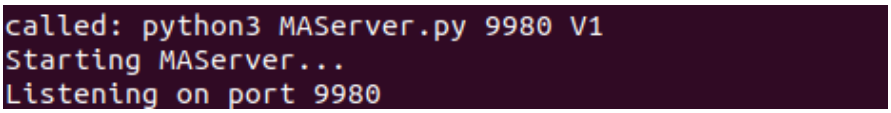


Figure 58: Output of misbehavior detection on local server port 9980

Finally running the scenario and choosing the following scenario and technologies:

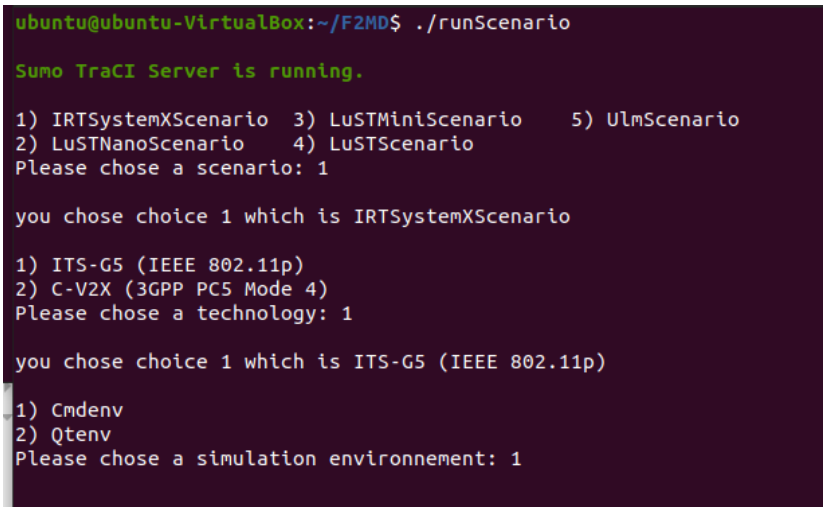


Figure 59: Launching the Paris-Saclay scenario using IEEE protocol and CMdenv

After launching the simulation, the sumo gui opens up showing a graphic visualization to the simulation:

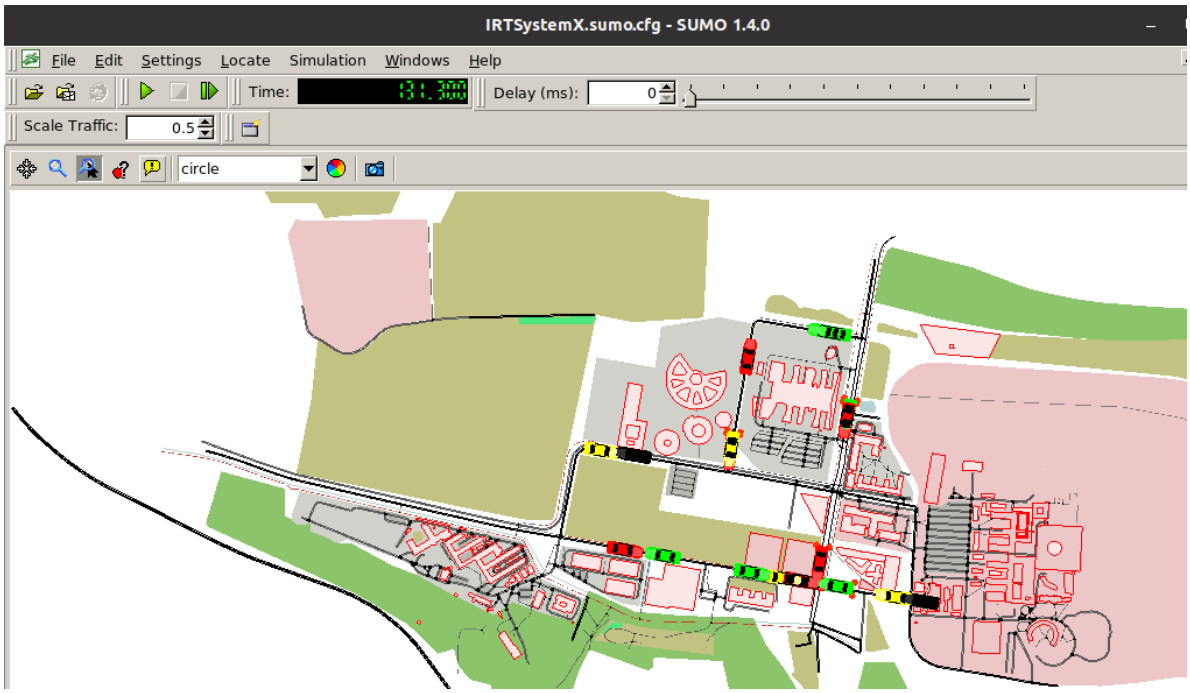


Figure 60: SUMO Simulation graphic output

We can observe the color differences of vehicles as:

- green refers to benign vehicles
- red refers to attacking vehicles
- yellow refers to Sybil nodes

5.2.3.2. Detection results

On the browser, we can watch the detection rate which evolves as long as the simulation continues at a periodic time thanks to every analyzed reports sent during the ongoing simulation under a periodic time:

The misbehavior server uses a local server under the port 9980 thus we have

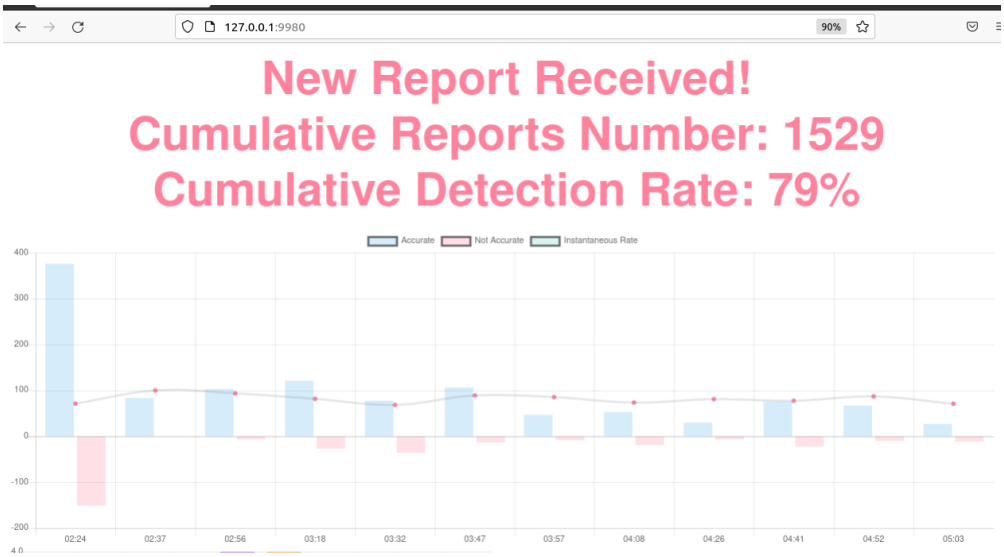


Figure 61: Misbehavior detection server output

With the help of reaction register graphic:

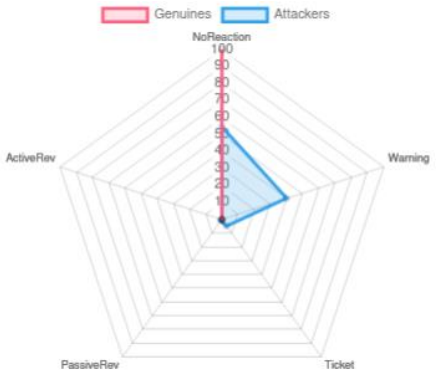


Figure 62: Reaction of ongoing attack reports

The results of this simulation are recorded in the file “f2md-results” under the “IRTSystemXScenario-ITSG5” where we have the newly created VeReMi dataset:

```
ubuntu@ubuntu-VirtualBox:~/F2MD$ cd f2md-results/
ubuntu@ubuntu-VirtualBox:~/F2MD/f2md-results$ ls
gui-plots  IRTSystemXScenario-ITSG5  plotScenario  plots.jpg  README.md
ubuntu@ubuntu-VirtualBox:~/F2MD/f2md-results$ cd IRTSystemXScenario-ITSG5/
ubuntu@ubuntu-VirtualBox:~/F2MD/f2md-results/IRTSystemXScenario-ITSG5$ ls
AppV1.dat            mdaV1.dat
AppV1Flags.dat       mdaV2.dat
AppV1FlagsInst.dat   MDBsms_V1_2021-9-21_12:24:7
AppV1Inst.dat        MDReports_V1_2021-9-21_12:24:7
AppV2.dat            VeReMi_0_3600_2021-9-21_12:37:2
AppV2Flags.dat       VeReMi_0_3600_2021-9-21_12:55:39
AppV2FlagsInst.dat   VeReMi_0_3600_2021-9-21_13:30:53
AppV2Inst.dat        VeReMi_0_3600_2021-9-23_5:5:1
ubuntu@ubuntu-VirtualBox:~/F2MD/f2md-results/IRTSystemXScenario-ITSG5$
```

Figure 63: The VeReMi dataset

5.2.4. Proposed work

For this part, we made use of the existing server hierarchy and applied modifications on the python algorithm of the misbehavior authority server to improve the detection results:

5.2.4.1. Misbehavior Authority server presentation

The misbehavior authority server is composed of five files each with its own objective:

- MAMain: this is the main function used upon activating the server and stays listening waiting for sent reports of possible misbehavior during the simulation.
- MAServer: handle the responses and listening to the reports to analyze
- MAStorage: this is for registering each report with the needed identifications (reported ID, Misbehavior type,Time of generated report)
- MAStats: To register number of the detected validly and the one failed to detect for performance verification.
- WebGuiBuilder: To create the interface used to display, on the local server using certain port, the results of the detection performance.

5.2.4.2. Work Implementation

The proposed idea is to add a type of reputation system to each vehicle depending on the results of the report, for example, a vehicle which keeps on sending reports which are found to be a false detection may be considered a compromised report, an attacking vehicle sending fake reports hence these reports can be ignored. Just like that, a vehicle with a good reputation of identifying misbehavior, its reports are reassuring for the identification of the malignant vehicle.

For this to work, we declared a dictionary type variable under the name of “vehicle_stat” in the storage class(MAStorage.py file). The keys of this dictionary are the IDs of the vehicle that sent the report and the values (assigned 0 on default) increments or decrements depending of the validity of the report.

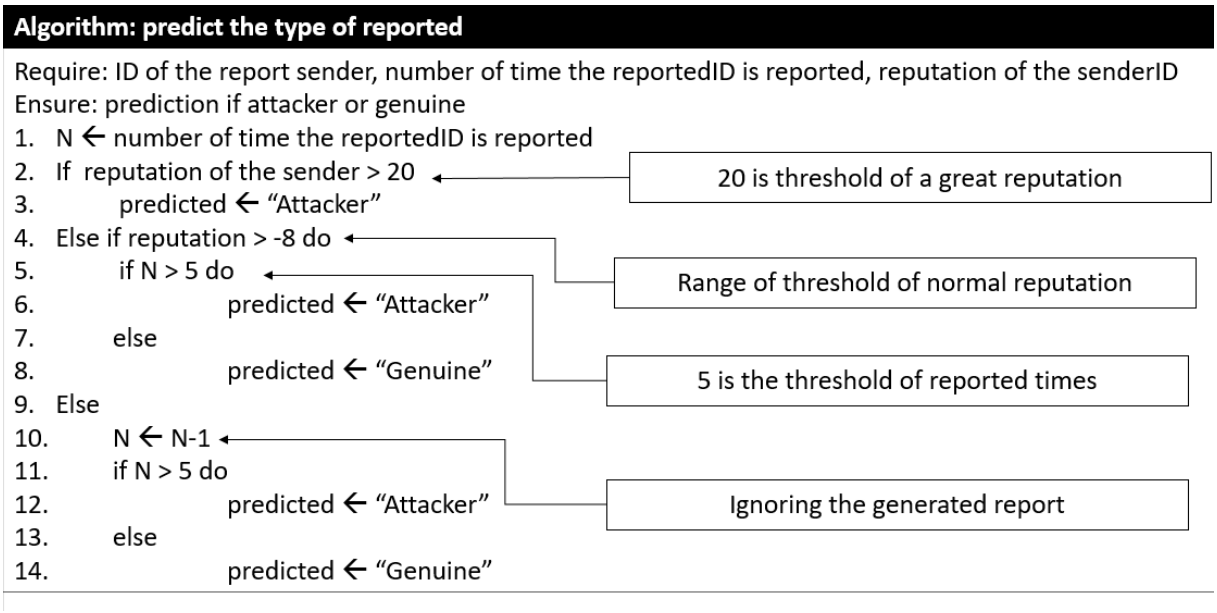


Figure 64: Pseudo algorithm of the prediction

Each report is sent on Json format from which we can read the reported, Time of generated report, the declared misbehavior and the sender pseudo ID. For each report, if it is declared an attack we verify the sender and if it has a great value on the "vehicle_stat" then the report is declared valid by default otherwise if the value is under the value 20 of threshold and is over the -10 a verification using a threshold on the number of reports on the vehicle which leads to valid report if it is greater than five reports and finally if the "vehicle_stat" is under the -10 than this vehicle is not trustworthy and such report is ignored.

After declaring the prediction, an update on the "vehicle_stat" is performed to increase or decrease the referring value to the sender ID. Applying this modification, we observe an impressive improvement to the Detection rate.

Since we can control the density of attackers, we tried it on:

- 25% attackers and 75% normal vehicles(first case)
- 50% attackers and 50% normal vehicles(second case)

Using the browse interface the detection rate on the first case is:

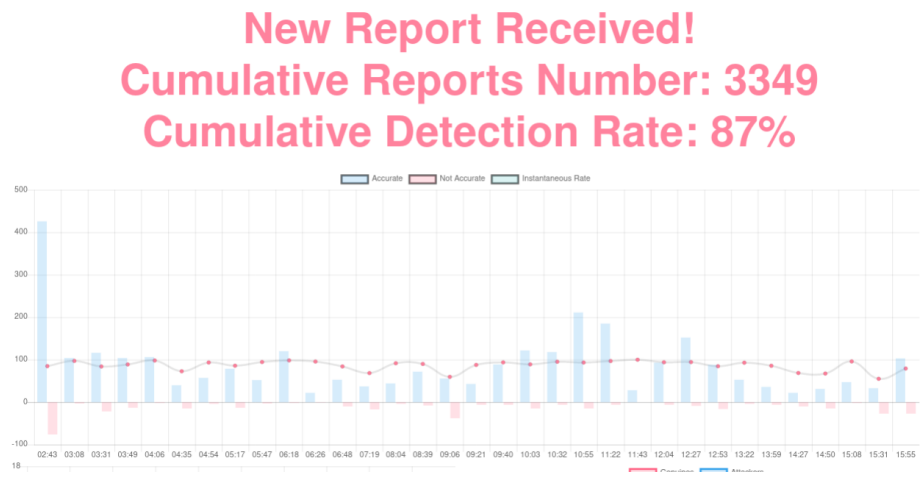


Figure 65: Misbehavior authority serve performance (first case)

Using the "./plotscenario" command on the simulation in work we can extract these results:

Confusion Matrix:	Predicted label 0	Predicted label 1
Actual label 0	79%	21%
Actual label 1	1%	99%

Precision: 0.99816
Recall: 0.70317
F1-score: 0.82509
Accuracy: 0.85955

Figure 66: Confusion Matrix and performance metrics(first case)

On the second case: we have these results:



Figure 67: Misbehavior Authority performance (second case)

Here we present the corresponding confusion Matrix in addition to the used metrics:

Confusion Matrix:	Predicted label 0	Predicted label 1
Actual label 0	72%	28%
Actual label 1	1%	99%

Precision: 0.99914
Recall: 0.82487
F1-score: 0.90368
Accuracy: 0.8797

Figure 68: Confusion Matrix and performance metrics(second case)

5.3. Conclusion

In this chapter, we resorted to the use of the simulation environment of the VANET which was made possible thanks to the use of various open source projects: SUMO, OMNeT, F²MD module on the Veins framework.

After the setting up the environment, we took our time understanding the performance of the F²MD servers then to applying certain modifications to improve the performance of the proposed algorithm.

Final step was to run the scenario on two cases of attacking vehicle density to observe the final results, hence proving the improvement introduced in the Misbehavior Authority server.

General Conclusion

This assignment was divided into three primary sequences, we begin with a study on the VANET digital threats precisely on the Sybil attack, then we took inspiration from other related researches to develop and implement an algorithm to detect the Sybil attack in the VANET, finally the preparation of the VANET simulator environment.

During this research, we achieved our goal by developing a Machine learning algorithm trained and tested using the VeReMi dataset that resulted in an impressive improvement in time management and prediction performance using the F1-score because of the unbalanced data:

- F1-score 89%
- Accuracy 87%

The performed steps began by implementing various modeling techniques and machine learning methods to find the most notable features that can symbolize the behavior of the vehicle on matrices. Each matrix was used to derive a couple of eigenvalues to train the ML model. Finally, a comparison of the ML model performance between the use of eigenvalues and the brut dataset.

Even though the proposed method presented a valuable result, it also displays some shortcomings on the eigenvalues computation portion that was time-consuming, but this was observed because of the use of a simple end device (My computer), hence, this method presents no problems with employing more capable processing power.

Finally, we prepared the VANET simulation environment that was used to develop the VeReMi dataset. On the F²MD Misbehavior Authority server, we proposed the addition of a reputation algorithm to boost the performance of the F²MD module.

A better solution to improve the F²MD module is to integrate the modification on the C++ programs to implement the trained module of misbehavior detection on the

simulation environment to be performed on the level of each node, hence a collaborative detection environment between the VANET nodes of the Sybil Attack.

References

[1]. Arai, Kohei, Supriya Kapoor, and Rahul Bhatia, eds. *Proceedings of the Future Technologies Conference (FTC) 2020, Volume 3*. Vol. 1290. Advances in Intelligent Systems and Computing. Cham: Springer International Publishing, 2021. https://doi.org/10.1007/978-3-030-63092-8.

[2]. Ghaleb, Fuad A., Anazida Zainal, Murad A. Rassam, and Fathey Mohammed. "An Effective Misbehavior Detection Model Using Artificial Neural Network for Vehicular Ad Hoc Network Applications." In *2017 IEEE Conference on Application, Information and Network Security (AINS)*, 13–18. Miri: IEEE, 2017. https://doi.org/10.1109/AINS.2017.8270417.

[3]. Grover, Jyoti, Manoj Singh Gaur, and Vijay Laxmi. "Multivariate Verification for Sybil Attack Detection in VANET." *Open Computer Science* 5, no. 1 (December 7, 2015): 60–78. https://doi.org/10.1515/comp-2015-0006.

[4]. Gu, Pengwenlong, Rida Khatoun, Youcef Begriche, and Ahmed Serhrouchni. "Support Vector Machine (SVM) Based Sybil Attack Detection in Vehicular Networks." In *2017 IEEE Wireless Communications and Networking Conference (WCNC)*, 1–6. San Francisco, CA, USA: IEEE, 2017. https://doi.org/10.1109/WCNC.2017.7925783.

[5]. Kamel, Joseph, Farah Haidar, Ines Ben Jemaa, Arnaud Kaiser, Brigitte Lonc, and Pascal Urien. "A Misbehavior Authority System for Sybil Attack Detection in C-ITS." In *2019 IEEE 10th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, 1117–23. New York City, NY, USA: IEEE, 2019. https://doi.org/10.1109/UEMCON47517.2019.8993045.

[6]. Kim, Myeongsu, Insun Jang, Sukjin Choo, Jungwoo Koo, and Sangheon Pack. "Collaborative Security Attack Detection in Software-Defined Vehicular Networks." In *2017 19th Asia-Pacific Network Operations and Management Symposium (APNOMS)*, 19–24. Seoul: IEEE, 2017. https://doi.org/10.1109/APNOMS.2017.8094172.

[7]. Lim, Kiho, Tariqul Islam, Hyunbum Kim, and Jingon Joung. "A Sybil Attack Detection Scheme Based on ADAS Sensors for Vehicular Networks." In *2020 IEEE 17th Annual Consumer Communications & Networking Conference (CCNC)*, 1–5. Las Vegas, NV, USA: IEEE, 2020. https://doi.org/10.1109/CCNC46108.2020.9045356.

[8]. Quevedo, Carlos H. O. O., Ana M. B. C. Quevedo, Gustavo A. Campos, Rafael L. Gomes, Joaquim Celestino, and Ahmed Serhrouchni. "An Intelligent Mechanism for Sybil Attacks Detection in VANETs." In *ICC 2020 - 2020 IEEE International Conference on Communications (ICC)*, 1–6. Dublin, Ireland: IEEE, 2020. https://doi.org/10.1109/ICC40277.2020.9149371.

[9]. So, Steven, Prinkle Sharma, and Jonathan Petit. "Integrating Plausibility Checks and Machine Learning for Misbehavior Detection in VANET." In *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 564–71. Orlando, FL: IEEE, 2018. https://doi.org/10.1109/ICMLA.2018.00091.

[10]. Heijden, Rens W. van der, Thomas Lukaseder, and Frank Kargl. "VeReMi: A Dataset for Comparable Evaluation of Misbehavior Detection in VANETs." *ArXiv:1804.06701 [Cs]*, April 18, 2018. http://arxiv.org/abs/1804.06701.

[11]. Kamel, Joseph, Michael Wolf, Rens W. van der Hei, Arnaud Kaiser, Pascal Urien, and Frank Kargl. "VeReMi Extension: A Dataset for Comparable Evaluation of Misbehavior Detection in VANETs." In *ICC 2020 - 2020 IEEE International Conference on Communications (ICC)*, 1–6. Dublin, Ireland: IEEE, 2020. https://doi.org/10.1109/ICC40277.2020.9149132.

[12]. Rajkumar, M Newlin, M Nithya, and P HemaLatha. "OVERVIEW OF VANET WITH ITS FEATURES AND SECURITY ATTACKS" 03, no. 01 (2016): 6.

[13]. Chawla, Appan, Rajeev Kumar Patial, and Dilip Kumar. "Comparative Analysis of Sybil Attack Detection Techniques in VANETs." *Indian Journal of Science and Technology*, n.d., 8. (2016)

[14] Douceur, J.R.: The Sybil attack. In: Proceedings of the InternationalWorkshop on Peer to Peer Systems, pp. 251–260 (March 2002)

[15]. Mishra, Debahuti, Buyya, Rajkumar, Mohapatra, Prasant, and Patnaik, Srikanta. *Intelligent and Cloud Computing Proceedings of ICICC 2019*, *Volume 1*, 2021.

[16]. https://veremi-dataset.github.io/ (2020-2021)

[17] "Microscopic Traffic Simulation using SUMO" ; Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner, and Evamarie Wießner. IEEE Intelligent Transportation Systems Conference (ITSC), 2018